7.2 Project 2: Next Track: A Music Recommender API

(Github : https://github.com/EiMyatMonPhyo/final_project_repo)
(Website hosted at : https://eimyat.pythonanywhere.com/)
(9785/ 10500 words)

Table of Contents

Introduction(810 /1000 words)

Recommender systems are becoming useful as there is a rapidly growing volume of data available today and it is hard to manually search the one you want from the vast amount of data. Therefore, today's music streaming platforms implement different recommender systems as part of their product to offer better user experience. By analyzing different characteristics of music or user behaviours, and preferences, the recommender systems can suggest tracks that match the user's taste and listening habits. This report focuses on the design, development and evaluation of a music recommender system that suggests a track similar to a user-provided list of tracks and preferences using content-based filtering.

The motivation of this project is to recommend a track based on the user inputs without using user behaviour data, listening histories or user tracking. Many existing recommender systems use content-based filtering, collaborative filtering or a hybrid of both. While the collaborative filtering approach can provide personalized recommendations with new discoveries, it relies on large amounts of user data. So, it lacks privacy and transparency. Moreover, it also has the cold start problem and, therefore, the system does not work well when dealing with new users or new tracks without previous data. In contrast, content-based recommendation systems can generate recommendations based solely on the features or characteristics of a track, without utilizing users' listening histories or behaviours. Therefore, the content-filtering approach is particularly suitable for stateless systems. This project aims to create an effective music recommendation system, which is better than a random recommendation system, by using high-level audio features and similarity metrics, without relying on user tracking or listening behaviours. The project also uses evaluation metrics to check the quality of the developed recommendation model and compares the model with other baseline models to see how well the model performs. User surveys are also collected to obtain real user feedback, which is used to iteratively improve the recommender system and enhance the user experience.

The domain of the project is music information retrieval and recommender systems. In this project, the tracks are represented using the high-level audio features which are useful in music recommendation systems to find the similarity between two music tracks. The target users of the system include individuals who want to discover a recommendation that is similar to the set of songs they like, without giving access to their personal data or behaviours.

This project follows the "7.2 Project Idea 2: Next Track : A Music Recommendation API" template provided by the university. As described in the template, the project will be focusing on developing a stateless API that accepts a list of track ids and optional user preference as input and returns a recommended track id as output. Therefore, the core feature of the project is the recommender system and the main functionality lies in the

backend logic and API design. A simple frontend which allows users to interact with the system is also developed. Subjective and objective evaluation are also conducted to identify weaknesses and iteratively improve the system.

The recommender system uses content-based filtering. Each track is represented as a normalized and weighted vector constructed from high-level audio features such as danceability, valence, energy, instrumentalness, tempo, loudness and acousticness. Similarity between tracks is computed using metrics such as Euclidean or Cosine distance and the most similar track's ID is returned to the user as output. This approach does not need user accounts, login, user personal or behavioural data. The system can also accept optional user preferences as input. A simple frontend is developed with search tracks, add tracks, remove tracks, select preferences and recommend functionalities to search, add, remove the tracks, select the preferences, and get recommendation from the system. The user survey focuses on the recommendation quality, preference settings, and user interface. The objective evaluation process involves splitting the playlist into 80%-20% parts, feeding the 80% of the playlist into the system to get top-k recommendations which are matched with the remaining 20% to compute the recommender performance. A number of different playlists needs to undergo the evaluation to determine the overall evaluation result. The baseline models such as Euclidean model, Random-by-artist model and the Random model are also implemented to compare to the main Cosine model. This evaluation and comparison can determine if the main model performs better than the random recommendation model, and the difference between the performances of different systems with different similarity metrics can also be observed.

Overall, the project primarily focuses on the API development of a recommender system, using content-based filtering techniques, high-level audio features, and some similarity metrics to recommend a track similar to the user inputs. A frontend is implemented and different evaluations are also carried out to know the model performance, and iteratively improve the system. The project combines concepts from music information retrieval, recommender systems and web development to create a stateless recommender system.

Literature Review (1730 /2500 words)

As Niyazov et al. [2021] stated “Nowadays, the amount of information provided to the user by modern information systems considerably exceeds the amount, that humans can physically examine, evaluate, and find something that suites to their goals. Therefore, when we face with information services containing a huge amount of content, it is worth to build a recommendation system to solve this problem.” [p. 274], today’s growing data leads to the implementation of recommender systems to improve the user experience.

Eliyas and Ranjana [2022] described Collaborative Filtering (CF) and Content-Based Filtering (CBF) as the two principal approaches to the recommendation systems. CF predicts a user’s preferences by analysing ratings or behaviours from many other users, in contrast, CBF relies on item attributes, using feature similarity to generate personalised recommendations [2].
The typical workflow of many CF systems include finding the similar users of the target user and generating the recommendations based on their data [2]. On the other hand, CBF systems involve feature extractions, feature comparison and suggestion based on the similarity between the features of the two items [2].
According to Eliyas and Ranjana [2022], the comparison between the two approaches can be made as follows:

Collaborative Filtering	**Content-based Filtering**
Data from a large number of users are needed.	No collection of user data is needed.
User tracking is essential.	No user tracking is needed.
Cold Start problem exists. (New items or new users cannot be recommended until enough information is collected.)	Can recommend new items to new users without user interaction data.
New items are recommended with the help of patterns from the other like-minded users.	Overspecialization Problem exists. (Similar items are always recommended, limiting the discovery of new items.
No transparency of the workflow exists as it depends on other user’s patterns.	Transparency of the system workflow exists.

The comparison presented by Eliyas and Ranjana [2022] influences the choice of approach used in my recommender system. The project template states that no user tracking should be used and the recommendation should be generated from the user

input list of tracks. Therefore, CF becomes unsuitable for my project as it relies on a large amount of user data to identify the patterns of similar users to make recommendations. Moreover, CF has the Cold Start problem, meaning the system cannot operate well without enough user or track interaction data, however, CBF can still perform effectively as long as the features of the track data exist.

In contrast, CBF aligns with the project requirements because it relies only on the songs' features, and does not depend on other users' behavioural data. Moreover, the transparency of CBF can be useful in evaluating the accuracy of recommendations. For these reasons, the analysis in Eliyas and Ranjana [2022] justifies the choice of CBF as the core method for this project.

Prior work has shown that content-based recommendations using music audio feature vectors perform significantly better than random recommendations [1].
A study has described and compared the two distance measures: Low-level measures and high-level measures [3]. The low-level measures used extracted descriptors such as MFCCs and spectral energy bands, and measured similarity using metrics like Euclidean distance and Kullback-Leibler divergence [3]. The high-level measures used high-level semantic descriptors, inferred from low-level descriptors through classifiers, and measured similarity with metrics such as Cosine distance, Pearson correlation distance, Spearman's rho correlation distance, etc [3]. The result has shown that high-level features outperformed low-level descriptors [3].
This shows that high-level features, when paired with appropriate similarity metrics are more effective for recommender systems. Considering the good results of content-based recommender systems using high-level features and vector representations, my recommender system is implemented using high-level audio features and vectorization.

Audio Signal Analysis which includes Music Emotion Recognition (MER), Perceptual Features, Genre Classification and Instrument Detection is one of the most common methods used in CBF [4]. Zeng and Umrawal [2025] stated that recommendation systems using MER can provide personalized recommendation by adapting to a user's mood, activities, or listening context. Perceptual features perform as a large part of MER and are also described as a bridge connecting the gap between low-level signal analysis and human emotional perception [4].
Therefore, it can be said that emotion plays a big role in providing personalized recommendation and perceptual features can be considered as important features for MER.

Spotify, one of the mainstream streaming sites, uses perceptual features as part of its recommender system [4]. As shown in Figure 1 (adapted from Niyazov et al., 2021, originally from Panda et al. 2021), perceptual features such as energy, valence, and acousticness are the top three features that can effectively discriminate the songs [4].

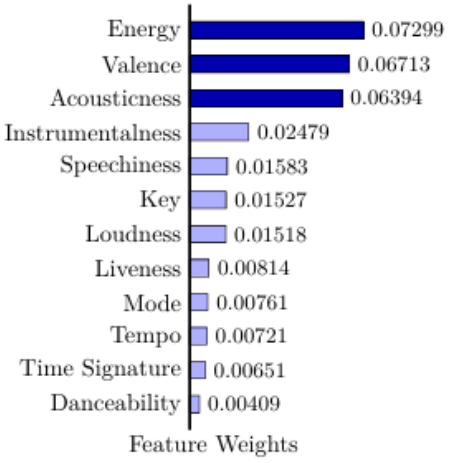


Figure 6: Usefulness of perceptual features to discriminate songs on arousal-valence axes, adapted from Panda et al. (2021)

Figure 1 Usefulness of perceptual features to discriminate songs, adapted from Niyazov et al., 2021, originally from Panda et al. 2021

Rhythm elements such as tempo can influence human emotion such as happiness or sadness [5]. Moreover, Spotify documentation describes that danceability is closely related to tempo, rhythm stability, beat strength, and overall regularity [6]. Therefore, danceability, together with tempo, has some influence on human emotion.
Moreover, as mentioned earlier, genre classification and instrument detection are part of audio signal analysis and closely related to MER [4]. There is also an association between loud music (high intensity) and intense emotion [5]. Therefore, genre, instrumentalness and loudness can have some impact on human emotion.
These relations between high-level audio features and human emotion are important factors to account for in music analysis and similarity. For these reasons, I use these features in my project to enhance the personalized recommendation.
For similarity metrics, Bogdanov et al. [2009] used Euclidean for low-level descriptors, and Cosine similarity and other metrics for high-level descriptors [3]. Niyazov et al. [2021] used distance metrics such as Euclidean, Manhattan and Cosine Distances on vectors of low-level extracted features and stated that Cosine Distance gives the most relevant recommendation [1].
It can be said that Euclidean and Cosine Distance are two of the most common similarity metrics in this field. Even though Cosine similarity provided the best recommendation according to Niyazov et al. [2021], their results were based on the

low-level features. Moreover, Cosine similarity focuses on the angle between the vectors while the Euclidean distance considers the features' magnitudes. Since my project relies mainly on high-level perceptual features where magnitudes matter for similarity, Euclidean distance was initially considered potentially more suitable. However, both metrics are commonly used in recommender systems and Cosine distance has demonstrated strong performance in prior research, both Euclidean and Cosine models were developed and evaluated. The better-performing model was then selected as the main model after development and evaluation.

Evaluation is essential and one of the most vital steps in machine learning tasks [1]. Therefore, it is necessary to do the evaluation process to check how good the implemented recommender model is performing.
Both objective evaluation, which uses metrics to measure recommendation quality, and subjective evaluation, which collects user ratings or feedback, are common approaches for evaluating content-based recommender systems [1]. In this project, I used objective evaluation to measure recommendation quality, while subjective evaluation through user feedback was also conducted to complement the quantitative results.
Since user perception is important in recommendation systems, a subjective evaluation through user surveys is very useful to gather user feedback regarding recommendation relevance and overall user experience.
In many papers, researchers have proposed various methods to evaluate the system with the objective approach [1,7].
Niyazov et al. [2021] proposed a method to use external characteristics i.e, the high-level track data, for evaluation process. The purpose was to take the number of matching characteristics between the recommended track and the input target track as a metric determining the quality of the system [1]. In the research by Niyazov et al. [2021], tags, identifiers for music description, were used for determining relevant recommendations. The tags of all the tracks used in the recommender system were downloaded, and a small part of the dataset was used for testing and the other part of the dataset was used in training the recommender system [1]. F1 score, calculated using the number of matching tags between tracks as a ground truth, was used to determine the quality of one recommendation by the system [1]. Overall evaluation was determined based on how many recommendations were relevant out of all recommendations [1].
On the other hand, the core material used for determining the quality of the recommender system in the research by Vall et al. [2019] was playlist. For the evaluation, the playlist was split into two: the larger one being the playlist to be inputted and the smaller one being the continuation list (i.e, the list of relevant tracks that should be recommended by the system) [7]. The system's recommended ordered list of songs was compared against the continuation list using the evaluation metrics such as

- Rank - which rank each song from the continuation list occupies in the ordered list of candidate songs,
- Reciprocal rank - the inverse of best rank song from continuation list in the ordered list of candidate songs, and
- Recall@100 - the number of continuation-list songs appearing in the top 100 recommended list [7].

Average values of all playlists were calculated for overall evaluation of the system [7]. Tags-based evaluation checks the quality based on music metadata similarity between the tracks while the playlist-based evaluation does it based on the user's playlist. Both approaches have strengths. Playlist-based evaluation is more user-oriented if the real user playlist can be used in the evaluation because it reflects the real listener behaviours and preferences. On the other hand, tag-based evaluation cannot capture the user perspective as it relies on the factual song descriptions for similarity, providing a consistent and objective result. We can consider tag-based evaluation as 'scientific' while playlist-based evaluation is more 'user-centric'. Both methods are good in their own way, however, considering that my system uses high-level data of the tracks to make recommendations, I chose playlist-based evaluation so that I will not be using the same material of high-level data of the tracks for both recommendation and evaluation processes.

The evaluation of a system was further performed by using random baselines in the paper by Niyazov et al. [2021] and Bogdanov et al. [2009]. This approach helps determine whether the developed model performs effectively compared to random recommendation systems. Additionally, evaluation can include a comparison between different similarity metrics to identify which performs best for the developed recommender system.

Design (1955 /2000 words)

Overview

This project focuses on designing and implementing a music recommender system which suggests a track that is similar to the user-provided list of songs. The system is built using vectors representing the high-level audio features provided by Spotify, and recommendations are generated through vector similarity computations using Cosine Similarity. The project can showcase web-development architecture with machine learning algorithms, where the main deliverable is a functional API that receives a list of track ids and user preferences as input and returns a similar track's ID. The main task of this project focuses on the backend and API implementation, model algorithm and logic, and the evaluation of the developed model. Besides that, a simple frontend interface for user interaction is implemented, too. The aim of the project is to implement a content-based recommender system that is better than the random selection, without requiring personal user data or behavioral tracking.

Template Used

The project uses the "7.2 Project Idea 2: NextTrack: A music recommendation API" template provided by the university and follows the instructions and rules stated in the template.

Domain & Users

The project lies within the domain of music information retrieval and recommender systems. It specifically focuses on content-based music recommendation, where the recommendations are generated from high-level audio features provided by Spotify rather than user's behavioural data and user tracking data. The target users are the people who want to discover a track similar to their list of songs without sharing their personal data or behavioural history.

Design Choices based on user needs and domain requirements

The design of the recommender system is justified by both the need of users and domain requirements. As described in the template, the main task of the project is to implement a stateless API which recommends the users a similar song to the user-provided list of tracks without user tracking. This motivates the use of a content-based filtering approach with Cosine Similarity, as it allows the system to compute song similarity directly from high-level audio features without user tracking or

requiring the user to provide personal information or long-term user behavioural data. So, this design supports user privacy and statelessness by providing instant recommendations without user's past data or behavioural tracking.
Thinking from domain aspects, together with collaborative filtering, content-based filtering is one of the common approaches partly used in many recommender systems. However, to satisfy the template instruction or user need which says to implement a stateless API without user tracking, only the content-based filtering approach is used to build the recommender system. High-level features are used as it is computationally more efficient and shows better results than the low-level features [3]. High-level features (danceability, energy, valence, acousticness, tempo, instrumentalness and loudness), provided by Spotify, are carefully selected based on factors such as emotion and the ability to discriminate the songs as described in Literature Review. The most common similarity metrics used in comparing the tracks are Cosine Similarity and Euclidean distance [1,3]. Cosine Similarity is used as a similarity metric to find the closeness between the two vectors in the vector space as it gives more relevant recommendations than Euclidean distance according to many research papers.
As said in the template, the project's main focus is the implementation of backend API for the recommender system. A frontend interface is also added to allow users to search tracks, input a list of tracks, choose preferences, and receive recommendations interactively, but the core functionality remains in the API implementation.
Among many objective evaluation methods such as using tag-based, feature-based and playlist-based evaluation, playlist-based evaluation is used as it avoids circular evaluation in this project and better reflects user behaviour, thereby supporting personalized recommendations. Baseline models were also implemented for comparison with the main model. In addition, user surveys were conducted to collect real user feedback for improving system usability and recommendation quality.
Overall, the project is designed based on user needs and domain requirements, following the template instructions provided by the university.

Structure

Data Collection & Preparation

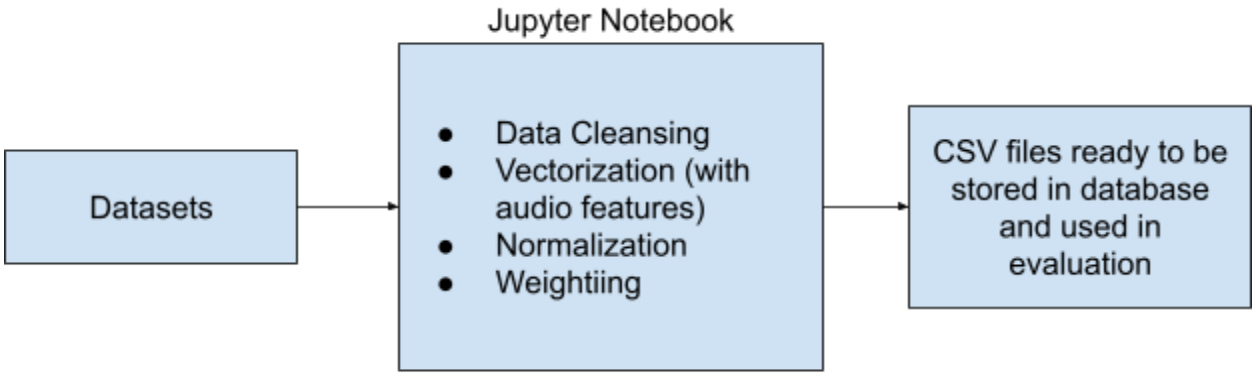
The data was collected from Kaggle's '30000 Spotify songs' dataset. Another music dataset 'Spotify top hit dataset' was also considered, however, it mainly contained annually curated popular songs influenced by mainstream trends. In contrast, the '30000 Spotify songs' dataset includes tracks grouped by genres across multiple playlists, making it more suitable for evaluating the recommender system.

Since there were 30000 tracks, only the playlists which include 15 to 40 tracks were selected for my project. Data cleansing such as dealing with missing values and removing duplicate tracks in the tracks dataset was performed. Audio features were added to a list to represent a vector. Data preparation was also performed as follows.

Normalization : Normalization was done to all the audio features using the min-max normalization so that all features are fairly spread out across the dimensions in the vector space.

Weighting : After normalization, weighting was also applied to the features by multiplying the values of these features and their corresponding weightings.

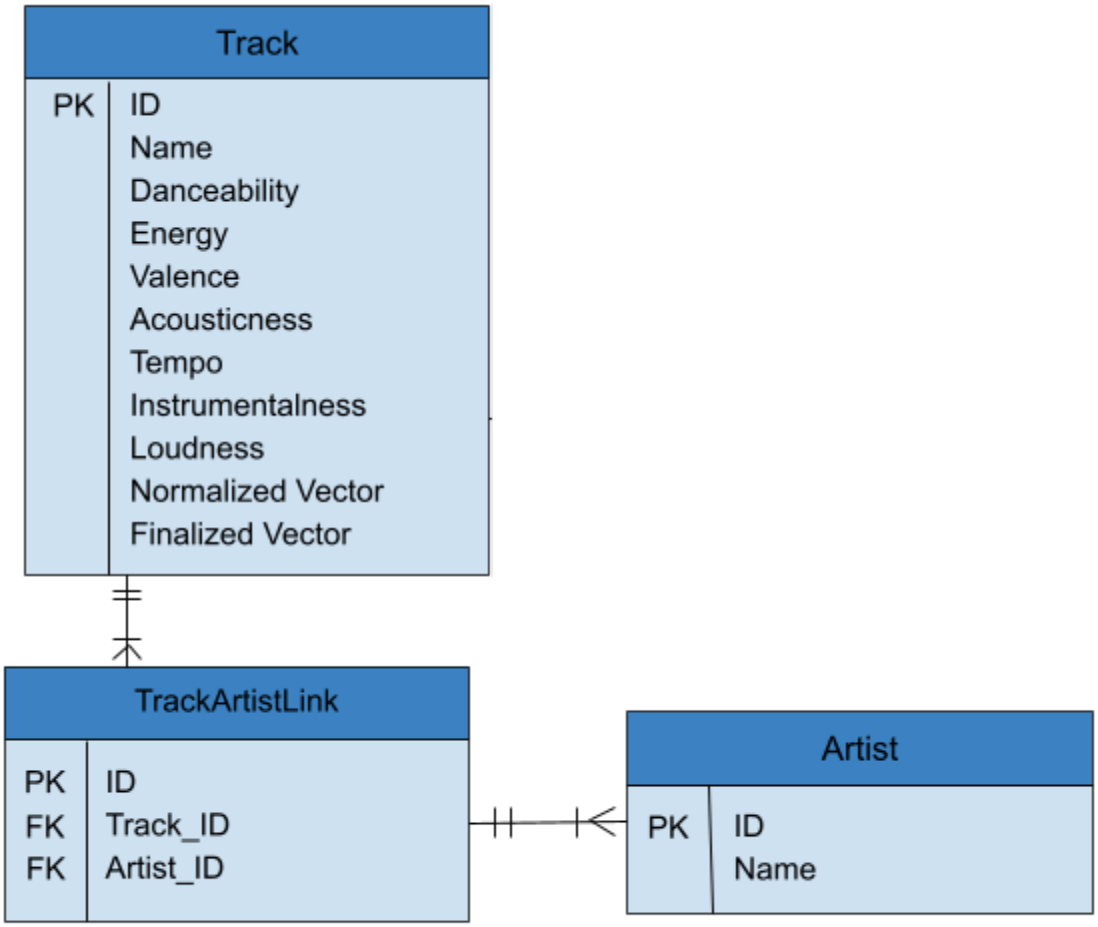
After cleaning the data and normalizing and weighting the vectors of audio features, the data was ready to be used and stored to the database. The playlists and their corresponding tracks were also stored in a separate CSV file for evaluation.



Data Processing in Jupyter Notebook

Database Design

The database includes 3 tables : Track, Artist and TrackArtistLink. The tables and their corresponding data are separated as follow:



Entity Relationship Diagram

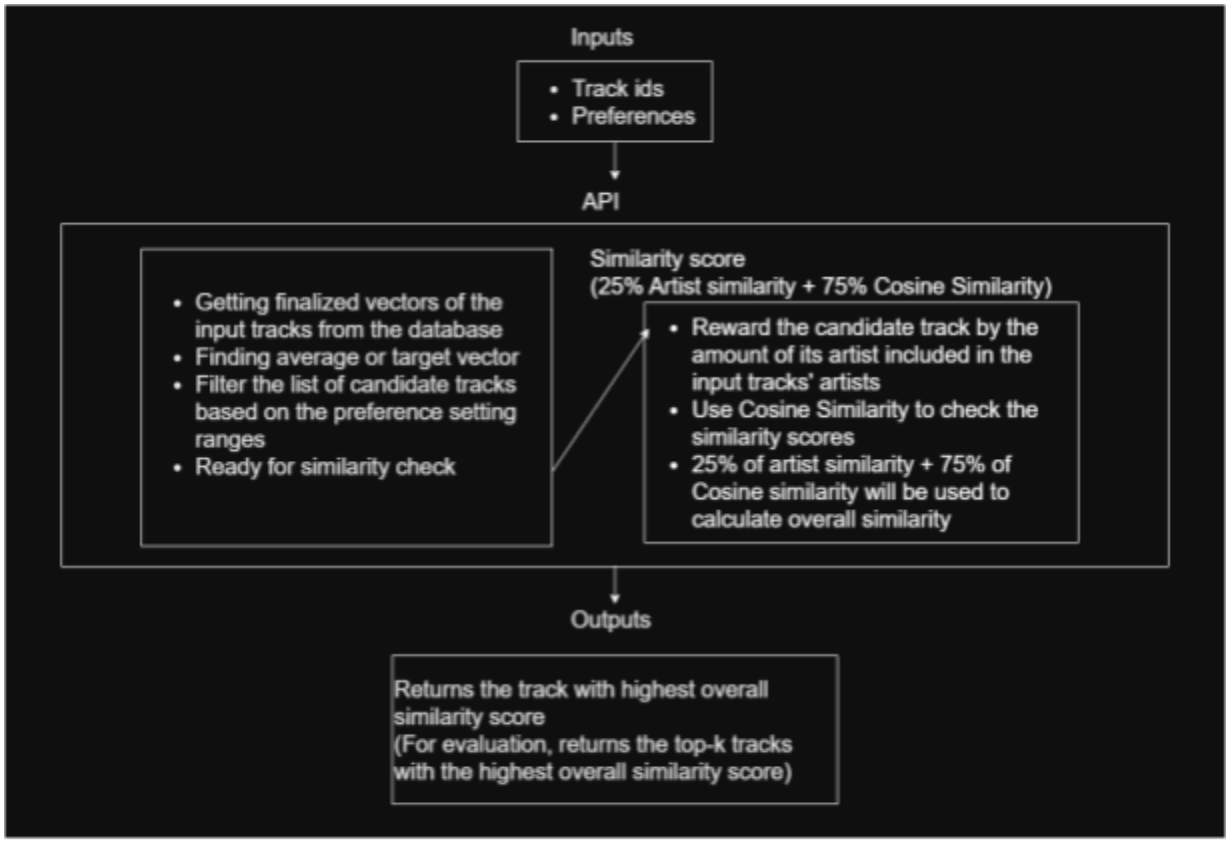
Recommender Logic

The user input, as stated in the template, consists of the list of track ids and optional user preference. The input is passed through the system's API endpoint or frontend interface, and the input track IDs are used to find their corresponding finalized vectors in the database. The average vector is computed to form the target vector.

Energy and tempo are chosen as optional user preferences as they are familiar to most users unlike other features like loudness and instrumentalness. The preference inputs are accepted as 'High', 'Medium' or 'Low', with respective ranges defined for each input. Based on the preference input, the tracks are filtered to get the eligible tracks to be compared to the target vector. Input tracks provided by the user are excluded from the candidate pool to prevent recommending songs already selected by the user.

For each eligible track, Cosine Similarity and the rate of artists included in the input tracks' artists are calculated, to get the overall similarity of each track with 25% of artist similarity and 75% of Cosine Similarity. The system will recommend the track with

maximum overall score. For evaluation, top k tracks are returned, where k is specified as a query parameter .



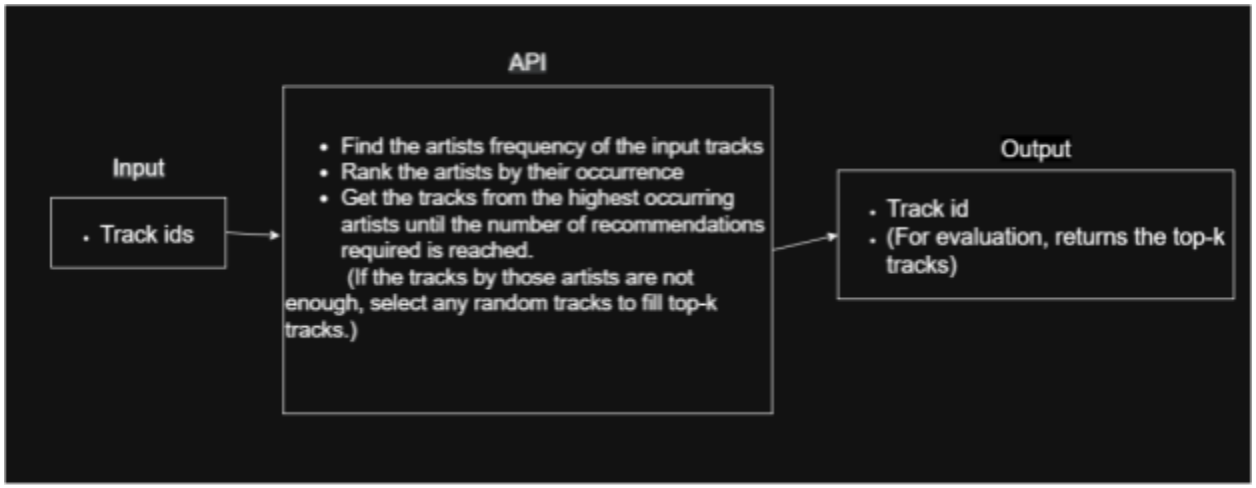
Structure of main recommender model (COSINE model)

Baseline models logic

Random model : Developed solely for evaluation purposes, this model ignores user preferences, and returns randomly selected tracks from the database, serving as a simple baseline for comparison.

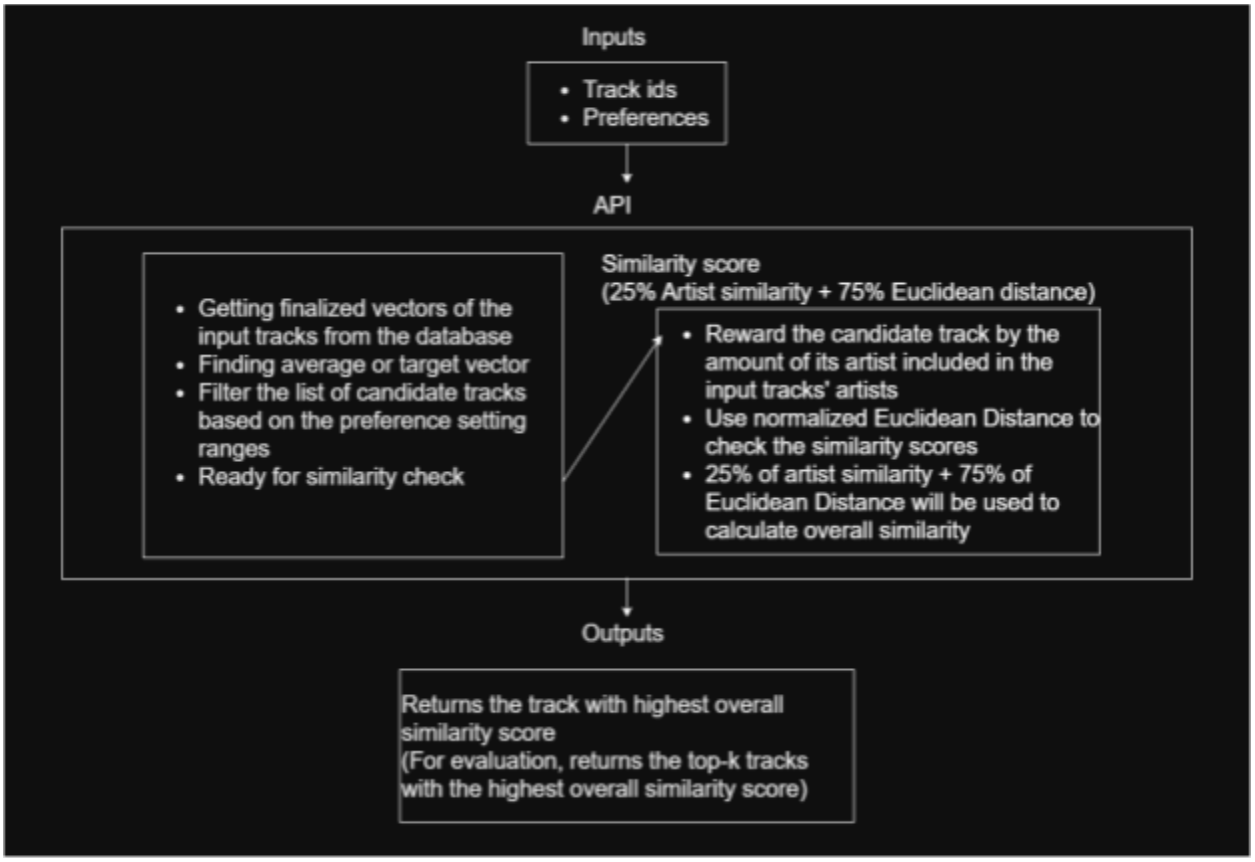
Structure of Random-by-artist baseline model

Random-by-artist model : this model retrieves artists associated with the input tracks and ranks them by frequency. Tracks from the most frequent artists are randomly selected until the required number of recommendations (top-k) is reached. If the available tracks of all ranked artists are insufficient, remaining recommendations are randomly filled. User preferences are not considered.



Structure of Random-by-artist baseline model

Euclidean model : This model follows the same process as the main recommender system but replaces Cosine Similarity with Euclidean distance as the similarity metric. Euclidean distance values are min-max normalized and converted to similarity scores using (1 - normalized scores) before combining with artist similarity to produce the final ranking.



Structure of Euclidean Baseline Model

Frontend interface

A simple frontend with a search bar, a display area of available tracks to be selected and add track buttons for each of them, another display area with selected input tracks and their remove track buttons, a display area for recommended tracks, a recommend button and the preference inputs are built. A music player and the Prev/ Next buttons are also implemented.

high fidelity Prototype

Technologies and Methods

Programming languages & Frameworks

Python – one of the most common languages in ML related projects, chosen for backend development and its extensive data-processing libraries. It is used to process data, implement recommender logic and similarity computations.
Django Framework – Chosen for building the backend API because it provides structured and scalable architecture. and my previous experience with Django allows for efficient implementation.

Frontend Technologies

HTML, CSS & Javascript, AJAX – chosen to implement an asynchronous website with a simple UI. Complex frameworks are not required because the project includes a simple frontend and mainly focuses on the backend and logic.

APIs

Spotify Web API – used to fetch required track and artist data.

Data Processing Tools

Jupyter Notebook – used to clean, prepare and process the data, especially converting audio features to normalized and weighted vectors, and used for evaluation.
Pandas – used for data processing
Numpy – used for vector operations such as Euclidean, Cosine similarity and other numerical operations.
Sklearn – used for data splitting and min-max normalization.

Database

SQLite – used to store the track and artist data. It is also compatible with Django framework.

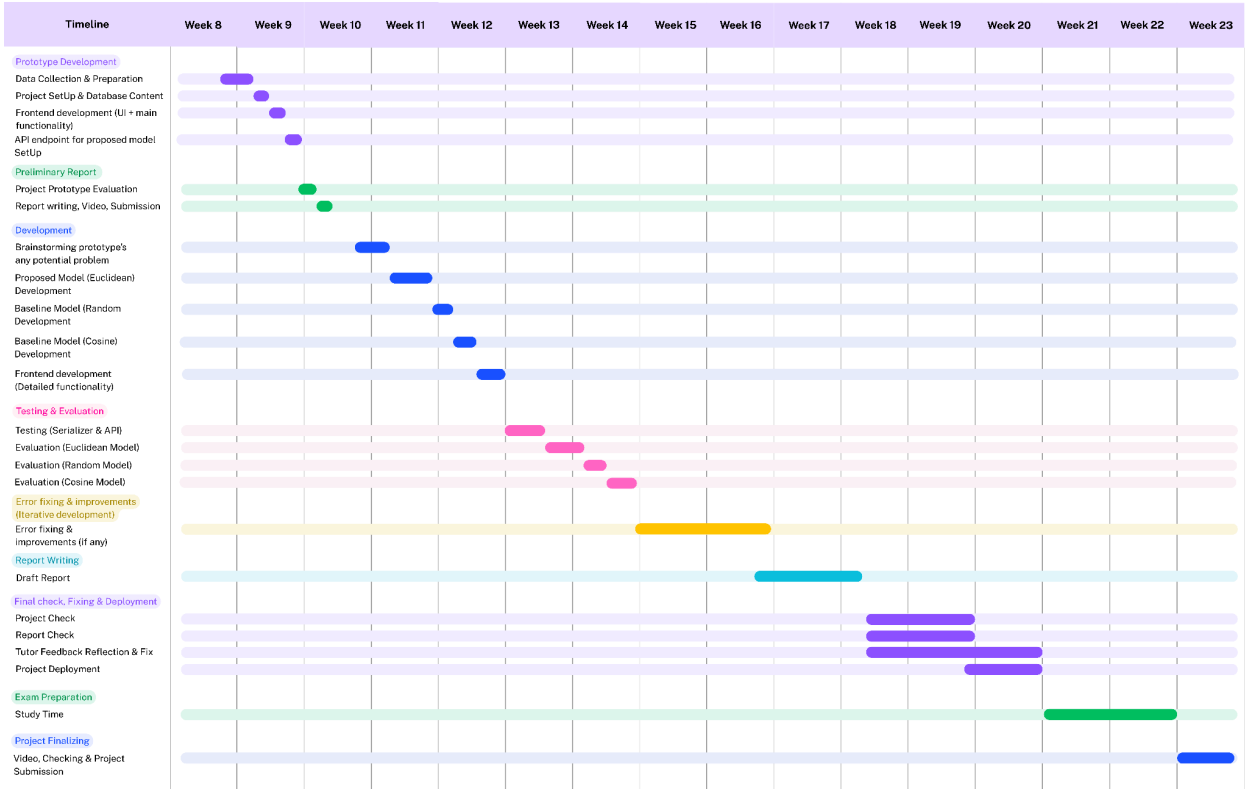
Recommender system methods

Vector representation – Each track is represented as a vector constructed from selected high-level audio features.
Normalization – Vectors are normalized to scale features to a standard range to avoid domination by different feature ranges. Euclidean scores are also normalized.
Feature Weighting – Weighting is applied to highlight features that are more discriminative for music similarity.
Cosine Similarity & Euclidean Distance – Used as similarity metrics as they are common metrics used in recommendation systems.

Evaluation

Playlist-based Evaluation – A subset of a playlist is used as input to the system and the top-k output tracks are compared with the remaining part of that playlist to evaluate the recommender performance.
User survey – A user survey was conducted to collect feedback on both recommendation quality and frontend usability. Users were asked to rate the relevance of recommended tracks and the frontend's user-friendliness. The collected feedback was used to improve the system during iterative developments.

Project timeline



Evaluation Plan

Playlist-based evaluation is carried out in the Jupyter Notebook because the evaluation is not part of the model and storing playlist data in the project's database system just for evaluation is not suitable. Moreover, Jupyter Notebook is more useful for writing the report of the evaluation results rather than doing it in the Django project.
For evaluation, playlists are loaded from a csv file. For each playlist, tracks are split into 80%-20% using Sklearn's train_test_split.
Evaluation using playlists: For each playlist, 80% of the tracks are used as input to the recommender, while the remaining 20% are treated as ground truth. The recommender generates a top-k list of tracks, which is then compared against the ground truth to identify relevant recommendations. Precision@k, Recall@k and NDCG@k are calculated based on these relevancies.
These metrics are computed for all the playlists, and their average scores are used to evaluate overall system performance.
Playlist-based evaluation does not use user preferences as real playlists already reflect user taste and adding random user preference will distort the evaluation results.

Baseline Comparison

To effectively evaluate the system, a comparison to three other models is performed. The three models (described in the ‘Baseline models logic’ section) for comparisons are:

1. Random recommender : Model which returns any top-k random tracks.
2. Random-by-artist recommender : Model which returns the top-k random tracks of the most frequent artists in the input.
3. Euclidean-distance recommender: Model which uses the same process as the main model but with Euclidean distance metric.

The first two systems are considered to develop for the evaluation to compare the difference and effectiveness between them and the proposed model. The other system is developed to see the impact of two different popular similarity metrics on the recommender systems.

The evaluation described above is performed for all four models and the accuracy values are compared. If the main model outperforms the random model, it indicates meaningful recommendations. Between Euclidean and Cosine models, the better-performing one is selected as the main model.

Implementations (2309 /2500 words)

The implementation consists of

- data preparation,
- the database implementation,
- the API development,
- the development of different recommender models logic,
- the frontend development.

The system is developed using Python Django framework and RESTful API for backend implementation, SQLite for the database, HTML, CSS and Javascript for frontend user interface and the Jupyter notebook for data preparation and evaluation of the models.

Data Preparation

Data Selection

The dataset '30000 Spotify songs' from Kaggle was used to obtain the tracks and playlists data. Initially, playlists with 20 to 30 tracks were chosen from 30000 tracks but this reduced the total number of tracks to 436 tracks, which was insufficient for a music streaming site. Therefore, the playlist range was expanded to 15-40 tracks, resulting in 1315 tracks. This resulted in 12 to 32 input tracks and 3 to 8 ground truth tracks for 80%-20% data splitting in evaluation, which I considered as a good amount for the project evaluation.

Column Selection

The track name, ID together with audio features (danceability, energy, valence, acousticness, tempo, instrumentalness, loudness) were selected from the dataset and the corresponding artist ID and name were fetched using the Spotify API.

Text Processing

The dataset did not contain any missing values or invalid entries, but to improve the consistency of the track names, a set of regular expression patterns was used to remove noisy common suffixes or extra descriptive information. These cleaned titles were also stored as a new attribute together with the data selected above.

Normalization and Vectorization

Audio features were then normalized using MinMaxScaler to have an equal feature influence of 0-1. These normalized features were transformed into a vector, representing each track's position in vector space.

Weighting

After normalization, weighting was then applied. The weight vector was defined with higher weights for energy, valence and acousticness as shown in the image below. Each normalized feature vector was then multiplied element-wise by the weight vector in order to scale the contribution of each audio feature accordingly.

```python
weight_list = np.array([
        1.1,  # danceability
        1.4,  # energy
        1.3,  # valence
        1.3,  # acousticness
        1.1,  # tempo
        1.2,  # instrumentalness
        1.2   # loudness
])

# Weighting : multiply each normalized vector by its corresponding weight (numpyarray * numpyarray) -> convert back to python list
tracks["weighted_vector"] = tracks["normalized_vector"].apply(lambda norm_v: (np.array(norm_v) * weight_list).tolist())
```

Removing duplicates

Since some tracks occurred in multiple playlists and the same tracks should not be stored multiple times in the database, duplicate tracks were removed, keeping only unique tracks. This resulted in having 1310 unique tracks ready to be stored in the database.

Producing CSV file

Track details, high-level audio features, fixed track name, normalized vector, weighted vector, and artist information were stored in the Track table. The playlist ID, name, and track ID and name are stored in the Playlist table, without removing duplicate tracks unlike the Track table. The resulting tables were as follows:

	track_id	track_name	danceability	energy	valence	acousticness	tempo	instrumentalness	loudness	artist_id	artist_name	fixed_track_name	normalized_vector	weighted_vector
1497	4qEoqyPbLYnLOii6mKlljl	Determinate - From "Lemonade Mouth"	0.562	0.768	0.218	0.003610	139.968	0.000000	-5.006	[3dctbbXhrRgigX1icexnws, 4VhL8KLjVso4vLfOLVViT...	[Adam Hicks, Bridgit Mendler, Naomi Scott, Hay...	Determinate	[0.5376427541856083, 0.7679593928937565, 0.223...	[0.5914070296041691, 1.075143150051259, 0.2900...
1498	5lz0NiPw32Gq4kMIUJvuw2	Take On the World - Theme Song From "Girl Meet...	0.638	0.713	0.703	0.001030	115.967	0.000241	-6.475	[45af7IeC0N5gQ9cyolFyS6, 74KM79TiuVKeVCqs8QtB0B]	[Rowan Blanchard, Sabrina Carpenter]	Take On the World	[0.621909302583435, 0.7129497662090866, 0.7195...	[0.6841002328417786, 0.9981296726927211, 0.935...
1499	1rM0CnyUiiw6A9CHJRXjZA	Chillin' Like a Villain	0.697	0.775	0.827	0.031800	113.886	0.000000	-6.664	[7bp2ISdh12wcA8LyB1srfJ, 4uBD3fsHzb3IpOTGpicsb...	[Sofia Carson, Cameron Boyce, Booboo Stewart, ...	Chillin' Like a Villain	[0.6873267546291163, 0.774960618108169, 0.8464...	[0.756059430092028, 1.0849448653514364, 1.1004...
1500	744ZuzjXQmoJmOdk2l1ym9	Keep It Undercover - Theme Song From "K.C. Und...	0.688	0.774	0.803	0.361000	109.974	0.000000	-4.444	[6sCbFbEjbYepqswM1vWjjs]	[Zendaya]	Keep It Undercover	[0.6773478212662158, 0.7739604430775386, 0.821...	[0.7450826033928375, 1.083544620308554, 1.0684...
1501	6VcKAd94eVxNyudlWN91GH	Time of Our Lives - Main Title Theme From "I D...	0.467	0.817	0.647	0.000152	200.044	0.000594	-4.941	[188VINPQh5dyNfLUkevKqf]	[Olivia Holt]	Time of Our Lives	[0.4323095686883247, 0.816967969394644, 0.6622...	[0.4755405255571572, 1.1437551571525015, 0.860...

	track_id	track_name	playlist_id	playlist_name
1497	4qEoqyPbLYnLOii6mKlljl	Determinate - From "Lemonade Mouth"	7jQHOrErpLMStcUUSavQWR	Post teen pop
1498	5lz0NiPw32Gq4kMIUJvuw2	Take On the World - Theme Song From "Girl Meet...	7jQHOrErpLMStcUUSavQWR	Post teen pop
1499	1rM0CnyUiiw6A9CHJRXjZA	Chillin' Like a Villain	7jQHOrErpLMStcUUSavQWR	Post teen pop
1500	744ZuzjXQmoJmOdk2l1ym9	Keep It Undercover - Theme Song From "K.C. Und...	7jQHOrErpLMStcUUSavQWR	Post teen pop
1501	6VcKAd94eVxNyudlWN91GH	Time of Our Lives - Main Title Theme From "I D...	7jQHOrErpLMStcUUSavQWR	Post teen pop

These tables were exported as CSV files which were used to populate the system's database or evaluate the system.

Database

Database Modelling

The system uses Django's default SQLite database for data storage due to its simplicity and ease of setup. The database is modelled as described in the 'Design' section. The three tables - Track, Artist, and TrackArtistLink are implemented.
The 'Track' table includes :

- track_id,
- track_name,
- fixed_track_name,
- danceability,
- energy,
- valence,
- acousticness,
- tempo,
- instrumentalness,
- loudness,
- normalized_vector,
- finalized_vector

Only the 'track_id', 'track_name', 'fixed_track_name', 'energy', 'tempo', and 'finalized_vector' are essential for the system, but the other fields are included just in case they are helpful for evaluation and analysis. The track ID, and names are defined as character fields. The track_id is defined as a primary key with the maximum length of 22 as described in Spotify API documentation. The other audio features are defined as float fields while the two vectors are stored using JSON fields to preserve their list-based structure and allow them to be easily deserialized into numerical arrays during similarity computation.
The 'Artist' table includes :

- artist_id,
- artist_name

Both fields are character fields with artist_id being defined as primary key with maximum length of 22. The table is linked to the 'Track' table through the 'TrackArtistLink' table described below.
The 'TrackArtistLink' table has :

- track,
- artist

Each field represents the objects or rows of the above 2 tables. This table's corresponding rows would be deleted if the rows from the 'Artist' or 'Track' were deleted.

Database Population

The database was populated using the Python script.
The track data were stored in a dictionary of lists containing the track details. Artist data, converted into Python lists, were stored in a dictionary which maps artist ids to names. A

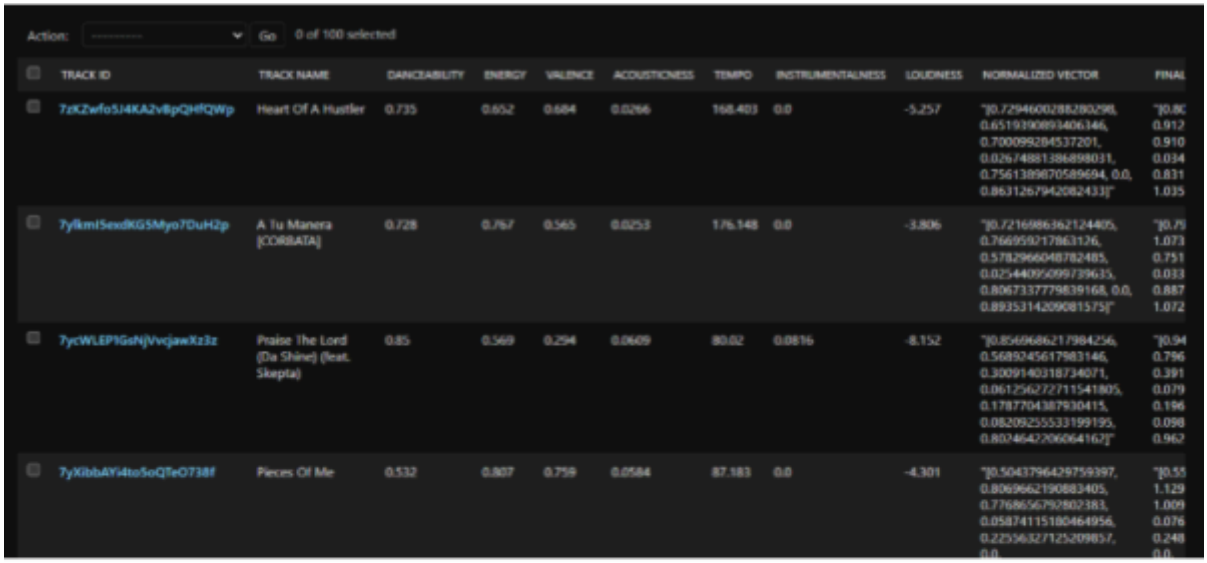
dictionary of sets was used to store the linked table's data, with track id being the key and the corresponding set of unique artist ids being the value.

```
tracks = defaultdict(list)
artists = defaultdict()
track_artists = defaultdict(set)

# read file
with open (track_data_file) as tracks_file:
    csv_reader = csv.reader(tracks_file,delimiter=',')
    header = csv_reader.__next__()

    for index, row in enumerate(csv_reader):
        tracks[index] = row[0:9] + row[11:14]       # col 0 to 8 and col 11 to 13

        artist_ids = ast.literal_eval(row[9])            # [id, id, id]
        artist_names = ast.literal_eval(row[10])         # [name, name, name]

        for i, id in enumerate(artist_ids):
            artists[id] = artist_names[i]           #{id: name}

        track_artists[index] = set(artist_ids)      # {index : {artistID1, artistID2}}
```

Rows for the three tables were created by looping those dictionaries and assigning values in them to corresponding fields as shown below. The 'TrackArtistLink' objects were created with references to the previously created rows of the other tables.

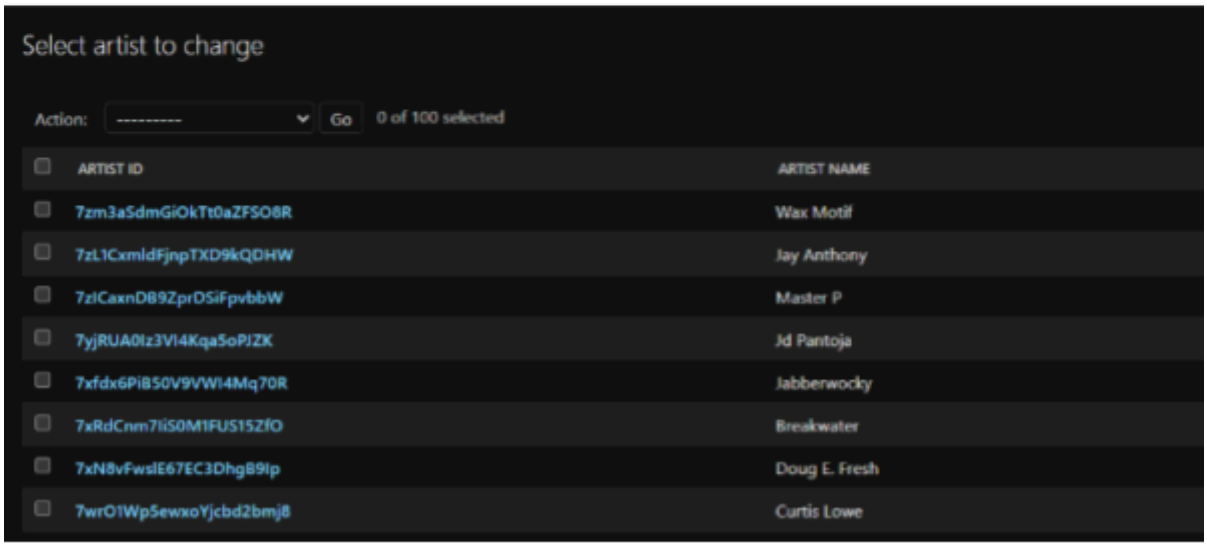
```
artist_rows = {}
track_rows = {}

# key (id), name (value)
for id, name in artists.items():
    row = Artist.objects.create(artist_id=id, artist_name=name)
    row.save()
    artist_rows[id] = row

for index, track in tracks.items():
    row = Track.objects.create(track_id=track[0], track_name=track[1], fixed_track_name=track[9],
                               danceability=track[2], energy=track[3], valence=track[4],
                               acousticness=track[5], tempo=track[6], instrumentalness=track[7],
                               loudness=track[8], normalized_vector=track[10], finalized_vector=track[11])

    row.save()
    track_rows[index] = row

for index, artist_ids in track_artists.items():
    for artist_id in artist_ids:
        row = TrackArtistLink.objects.create(track= track_rows[index], artist=artist_rows[artist_id])
        row.save()
```

Running the script populated the system database and the track data can be used in the system now.



Action: ---------- Go 0 of 100 selected

TRACK ID	TRACK NAME	DANCEABILITY	ENERGY	VALENCE	ACOUSTICNESS	TEMPO	INSTRUMENTALNESS	LOUDNESS	NORMALIZED VECTOR	FINAL
7zKZwfo5J4KA2vBpQHfQWp	Heart Of A Hustler	0.735	0.652	0.684	0.0266	168.403	0.0	-5.257	"[0.7294600288280298, 0.6519390893406346, 0.7000992845372 01, 0.026748813868980 31, 0.7561389870589694, 0.0, 0.8631267942082433]"	"[0.8 0.912 0.910 0.034 0.831 1.035
7ylkmISexdKG5Myo7DuH2p	A 1u Manera [CORRIDA]	0.728	0.767	0.565	0.0253	176.148	0.0	-3.806	"[0.7216988636212440 5, 0.7669592178631 26, 0.5782966048782485, 0.025440950997396 35, 0.8067337779839168, 0.0, 0.8935314209081575]"	"[0.75 1.073 0.751 0.033 0.887 1.072
7ycWLEP1GaNjVvcjawXz3z	Praise The Lord (Da Shine) (feat. Skepta)	0.85	0.569	0.294	0.0609	80.02	0.0816	-8.152	"[0.8569686217984256, 0.5689245617983146, 0.3009140318734071, 0.061256272711541805, 0.1787704387930415, 0.0820925553319919 5, 0.8024642206064162]"	"[0.94 0.796 0.391 0.079 0.186 0.098 0.962
7yXibbAYi4to5oQTeO738f	Pieces Of Me	0.532	0.807	0.759	0.0584	87.183	0.0	-4.301	"[0.5043796429759397, 0.8069662190883405, 0.7768656792802383, 0.058741151804649 56, 0.2255632712520985 7, 0.0,	"[0.55 1.129 1.009 0.076 0.248 0.0



Select artist to change

Action: ---------- Go 0 of 100 selected

ARTIST ID	ARTIST NAME
7zm3aSdmGiOkTt0aZFSO8R	Wax Motif
7zL1CxmldFjnpTXD9kQDHW	Jay Anthony
7zlCaxnDB9ZprDSiFpvbbW	Master P
7yjRUA0lz3Vl4Kqa5oPJZK	Jd Pantoja
7xfdx6PiB50V9VWl4Mq70R	Jabberwocky
7xRdCnm7liS0M1FUS15ZfO	Breakwater
7xN8vFwslE67EC3DhgB9Ip	Doug E. Fresh
7wrO1Wp5ewxoYjcbd2bmj8	Curtis Lowe

Populated tables in the Django admin site

Recommender Logic

Four different recommender logics were developed for the system and evaluation. The inputs of the recommender logic are the list of track ids, optional user preferences and number of recommendations to be returned and the output is a list of recommended track IDs.

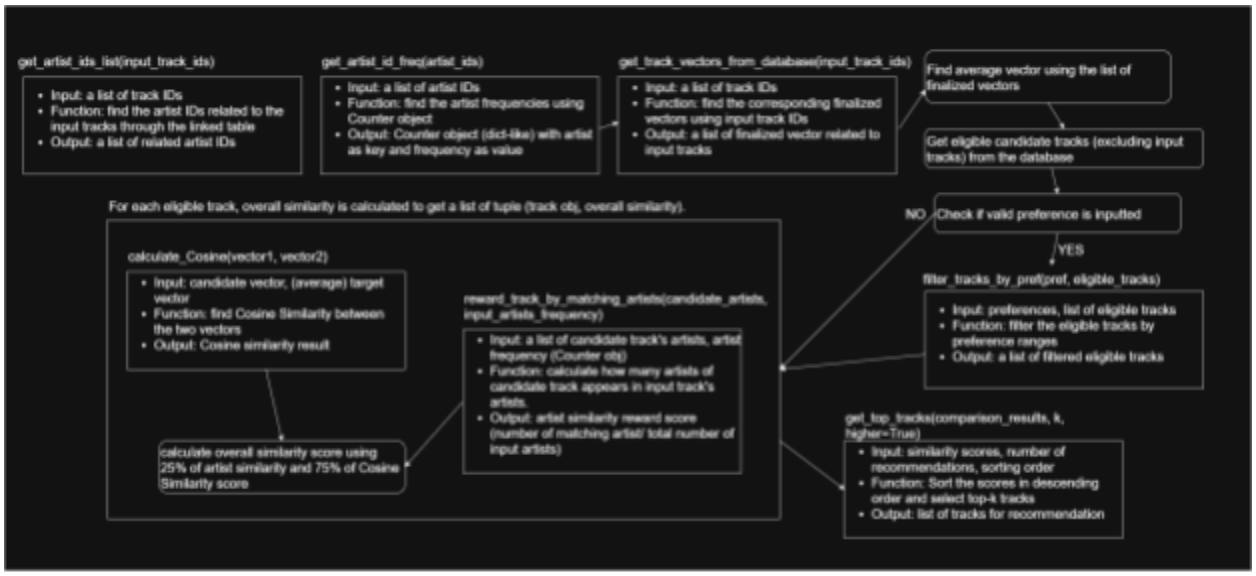
Cosine Model

1. Firstly, the recommender finds the input tracks' artists and calculates the artists' frequencies.
2. The recommender, then, takes a list of input track ids, and finds a list of corresponding finalized_vectors using the database operation.
3. Then, it calculates the average vector of those finalized_vectors using Numpy's mean function.
4. It then selects all the tracks stored in the database excluding the input tracks and filters the resulting tracks based on the input preferences, to get the candidate tracks.
5. Cosine Similarity is calculated using its formula and the artist similarity is calculated by getting how many artists of the specific candidate track are included in input tracks' artists. The overall similarity between the vector of candidate track and the average vector is calculated as 75% of Cosine Similarity and 25% of artist similarity.
6. Overall similarity scores are calculated for all the candidate tracks. The tracks and their respective overall similarity scores are stored as tuples in a list.
7. The similarity scores are sorted in descending order, and a list of top k tracks are selected from the sorted list and returned.

The default value of k is 1 and frontend uses k as 1. For API, query parameter 'k' can be added to the url for more recommendations.

Initially, recommendations relied only on feature-based similarity. After the first evaluation showed the strong performance of the Random-by-artist model, artist similarity was added as part of the overall similarity score in the second iteration.

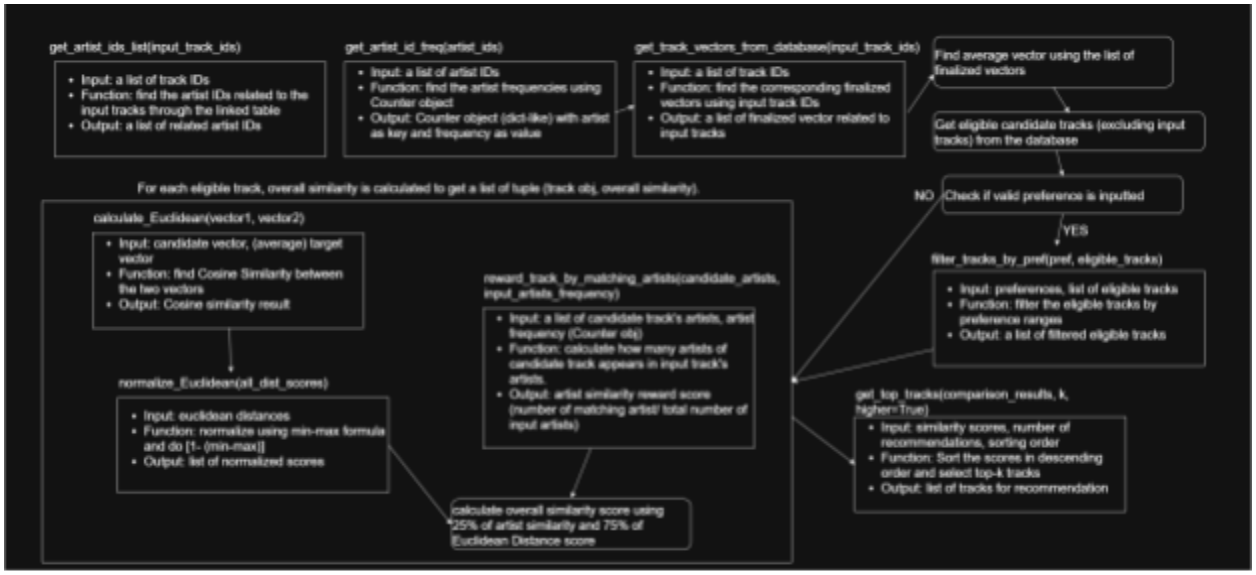
Moreover, before the first user survey, user preference was implemented by applying specific weightings to tempo or energy. Based on the survey feedback, current logic of filtering was implemented.



Implementation of Cosine Model Logic

Euclidean Model

The Euclidean model has the same steps and logic as the Cosine model but uses Euclidean distance as the similarity metric. Before combining with artist similarity to calculate the overall score, the distance values were normalized to a 0 -1 range and transformed using (1- normalized distance) so that higher values represent higher similarity.



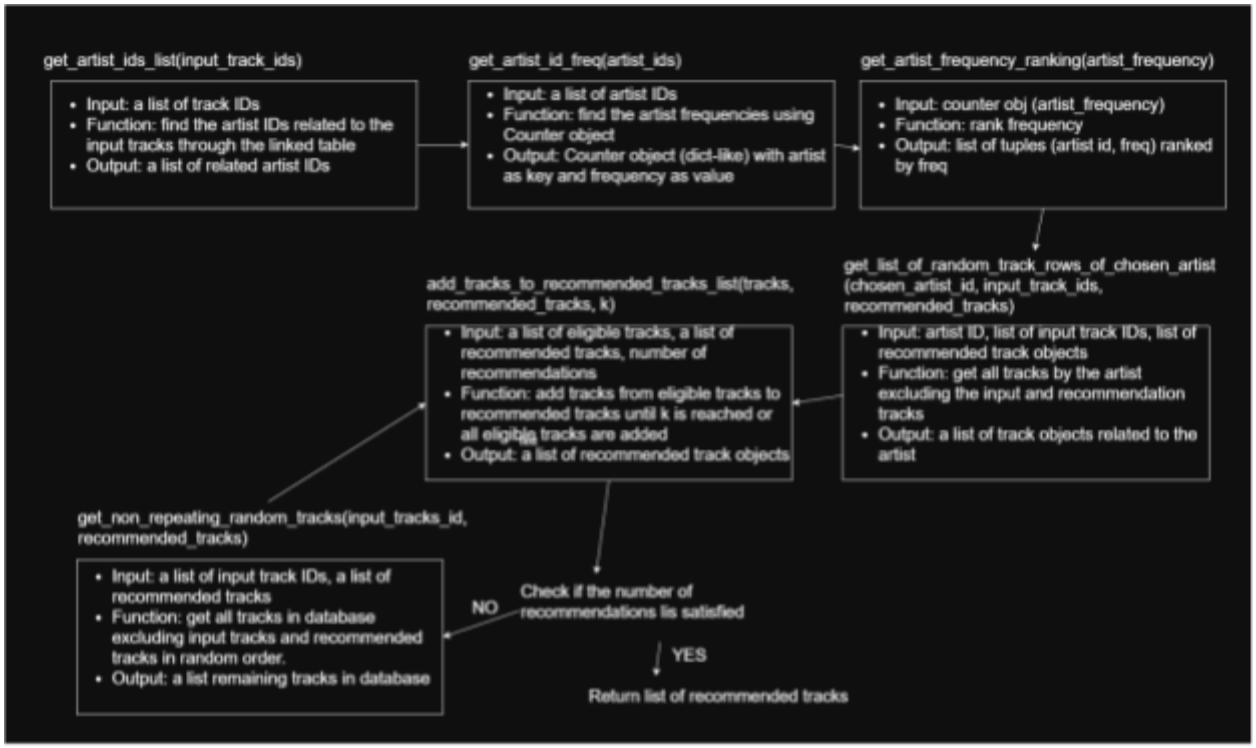
Implementation of Euclidean Model Logic

Random-by-Artist Model

The random-by-artist model was developed for evaluation purposes and does not use user preferences. It takes a list of input track IDs and the required number of recommendations.

1. Artist IDs of the input tracks are retrieved through database queries.
2. Artist frequencies are counted using Python's Counter object which returns a counter object (a dictionary-like structure with artist ID being the key and its frequency being the value).
3. Artists are ordered by descending frequency using the Counter's most_common method.
4. Tracks from the most frequent artists are retrieved first.
5. These tracks are shuffled to avoid returning the same order of tracks for the same artist, and are then added to the recommendation list. This process (Step 4 & 5) continues with artists of decreasing frequencies until the required number of recommendations is reached.
6. If the total number of tracks obtained from those artists is insufficient to reach the required number of recommendations, additional tracks are randomly selected to fill the output list.

Input tracks and already recommended tracks are excluded to prevent duplicates.

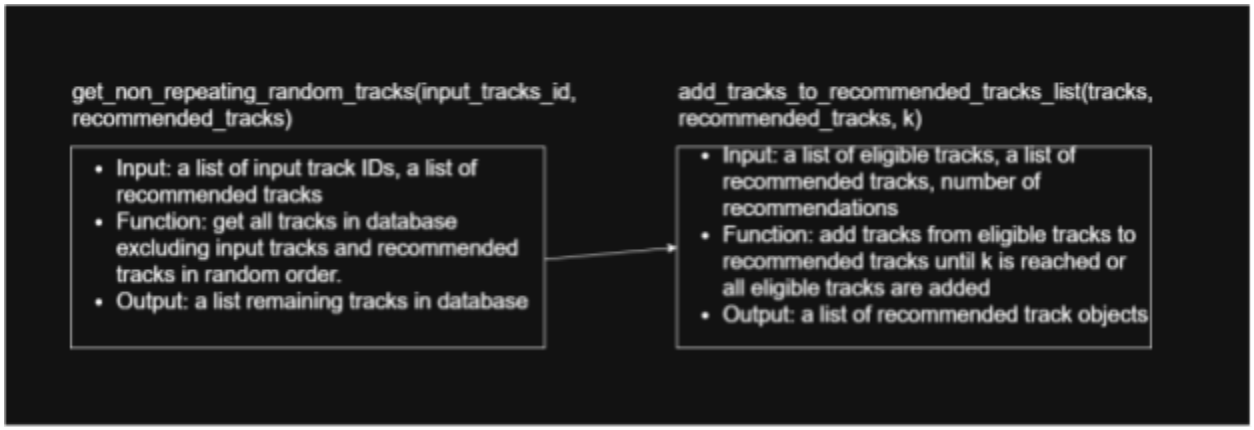


Implementation of Random-by-artist model logic

Random Model

Similar to the Random-by-artist model, this model, meant for evaluation, does not take preferences as input, but only takes a list of track IDs and the number of recommendations.

1. The random model returns randomly selected tracks from the database while excluding duplicate recommendations and input tracks.



Implementation of Random model logic

API implementation

Django Rest Framework was used for API.

Serializers

Serializers are used for data validation and conversion. API inputs include user preferences and a list of track IDs. Preference fields are defined as optional choice fields with values ‘High’, ‘Medium’ and ‘Low’. Track IDs are defined as a list of required 22-character fields with at least one element.
In the initial implementation, preference fields were defined as optional float fields ranging between 0.5 and 1.5 with a default value of 1.0, as preferences were applied through feature weighting. After the first survey indicated that preference settings were not effective, I changed the preference logic from weighting to filtering. Consequently, the serializer was updated to use categorical preference inputs as described above.

```python
class PreferenceSerializer(serializers.Serializer):
    choices = ['High', 'Medium', 'Low']
    energy = serializers.ChoiceField(choices=choices, required=False)
    tempo = serializers.ChoiceField(choices=choices, required=False)

class RecommenderInputSerializer(serializers.Serializer):
    track_ids = serializers.ListField(child=serializers.CharField(required=True, allow_blank=False, max_length=22),min_length=1)
    preferences = PreferenceSerializer(required=False)
```

The API output consists of track IDs, defined using the track_id field of the Track model.

```python
class TrackIdRecommendationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Track
        fields = ['track_id']
```

```
{
  "track_ids": ["2hBJgz4Ye9MkkmBbaDTTKx", "4xkOaSrkexMciUUogZKVTS","7pFydJbDEToJHtvl6g579k"],
  "preferences": {
    "energy": "High",
    "tempo": "Low"
  }
}
```

API input format

API output format for 1 recommendation

API output format for k:5 recommendations

API

A RESTful API endpoint was implemented to handle recommendation requests. The endpoint accepts a POST request containing a list of input track IDs and optional user preferences.
An optional URL parameter k is supported to control the number of recommendations returned, primarily used during evaluation. If no k value is provided, the default number of recommendations is one.
Request data are validated using the input serializers to ensure correct input formats. If preference is not provided, its value defaults to None.
Recommendations are generated using the recommender logic described earlier, and the output track IDs are serialized and returned to the client in JSON format.

```python
def make_recommendation(request, recommend_function, use_pref=True):
    try:        # k must be integer, other values are not allowed
        k = int(request.query_params.get("k",1))        #getting k for evaluation to be passed to recommender logic (1 for no pararms passed. k if param is passed)
    except:
        return Response({"error" : "The URL param top k 'k' must be an integer"}, status=status.HTTP_400_BAD_REQUEST)
    serializer = RecommenderInputSerializer(data=request.data)

    if k <= 0:
        return Response({"error" : "The URL param top k 'k' must be an integer greater than 0"}, status=status.HTTP_400_BAD_REQUEST)

    if serializer.is_valid():
        input_track_ids = serializer.validated_data.get('track_ids')


        try:
            if use_pref:        # for Euclidean or Cosine only (Other models are for evaluation purpose and don't need preference for that)
                input_preferences = serializer.validated_data.get('preferences')
                if input_preferences:        # if pref is inputted, get the data
                    input_preferences = {
                        "energy_input" : input_preferences.get('energy'),
                        "tempo_input" : input_preferences.get('tempo')
                    }
                else:      # if pref is not inputted, set pref to None
                    input_preferences = None
                track = recommend_function(input_track_ids, input_preferences, k=k)
            else:
                track = recommend_function(input_track_ids, k=k)

        except ValueError as e:
            return Response({"error": str(e)}, status=status.HTTP_400_BAD_REQUEST)

        # put output to serializer
        serializer_output = TrackIdRecommendationSerializer(track, many=True)
        return Response(serializer_output.data)
    return Response (serializer.errors, status= status.HTTP_400_BAD_REQUEST)
```

The same API structure is used for all recommender models, including the main model and baseline models, with only difference of underlying recommendation logic.

```python
# Main model : Euclidean
@api_view(['POST'])
def recommendTrackId(request):
    return make_recommendation(request, recommend_Euclidean_topk)


# Baseline : Cosine
@api_view(['POST'])
def recommendTrackIdCosine(request):
    return make_recommendation(request, recommend_Cosine_topk)

# baseline : random by artist
@api_view(['POST'])
def recommendTrackIdRandomByArtist(request):
    return make_recommendation(request, recommend_random_by_artist_topk, use_pref=False)

# baseline : random
@api_view(['POST'])
def recommendTrackIdRandom(request):
    return make_recommendation(request, recommend_random_topk, use_pref=False)
```

Frontend

The frontend of the system was implemented using Django views and HTML templates. Its primary purpose is to allow users to search for, add and remove tracks, adjust user preferences,

and receive recommendations by the recommender system. Session storage is used to maintain user inputs, preferences, and recommendations across requests.

Adding track

The frontend displays a list of tracks available with ‘Add Track’ buttons. When the user clicks a button, the clicked track’s ID is added to the session variable ‘input_tracks’.
Initially, the system was designed to accept track IDs entered manually and retrieve audio features through the Spotify API, allowing users to input any Spotify track. However, during the prototype development, I realized that the Spotify API endpoint for audio features was deprecated. Therefore, the system was redesigned to use only tracks stored in the system database.
Displaying available tracks also improved usability as users no longer need to know specific Spotify IDs. Following the design change, a search function was added to improve user experience.

Searching tracks

The search bar allows users to search tracks by artist name or track name. When the user enters a keyword, Django’s Q object filters tracks whose ‘fixed_track_name’ or ‘artist_name’ fields contain the keyword. Tracks that are already selected as input tracks are excluded from the results. The filtered results are rendered for user selection.
If no keyword is provided, all available tracks except selected input tracks are displayed.

Updating preferences

The preferences for energy and tempo are presented as drop-down options (‘High’, ‘Medium’, ‘Low’) instead of numerical weights to improve usability. The default value is None. When a user updates a preference, it is stored in the session variable ‘preferences’. The preferences are later converted into numerical ranges in recommender logic functions.

Removing track

The frontend displays a list of input tracks with ‘Remove’ buttons, and the user can remove any added track by clicking the corresponding ‘Remove’ button. The system then removes the selected track’s ID from the session variable ‘input_tracks’.

Getting recommended track

When the user clicks the ‘Recommend’ button, the system calls the Cosine recommender logic, passing the current session values of ‘input_tracks’ and ‘preferences’. The system then retrieves the track’s details (ID, name, and artist information) to store it in the session variable ‘recommended_track’ and display on the interface.

HTML template

The interface was implemented using HTML and CSS. User inputs are accepted using CSRF-protected forms. Adding and removing tracks, updating preferences, and retrieving recommendations are triggered through separate buttons and drop down options. A Spotify music player was also embedded using an iframe element, allowing users to listen to tracks directly within the system.
Javascript was implemented to manage the Prev and Next playback buttons. They were newly implemented after the second survey to improve usability.
A track history array stores the sequence of tracks on the music player, while a current index variable tracks playback position. Based on this history, navigation buttons allow users to move between tracks without backend requests. Button states are dynamically enabled or disabled based on navigation availability.

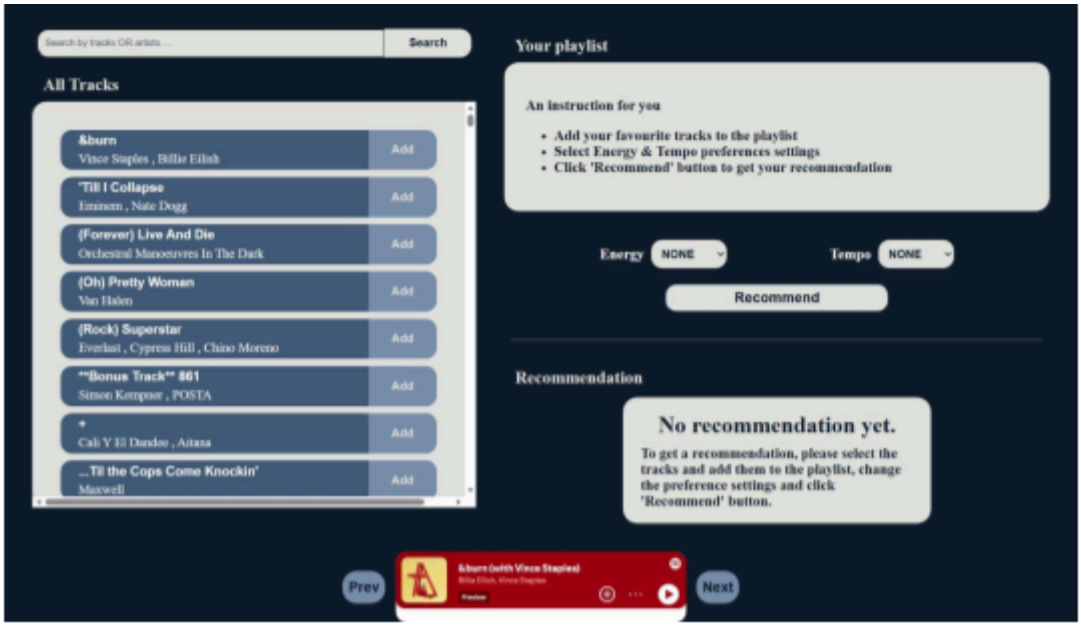
Asynchronous Interaction using AJAX and JavaScript

Initially, the system relied on standard HTTP requests, causing full page reloads after each interaction. Second user feedback indicated that this behaviour interrupted the listening experience because the music player restarts after each interaction.

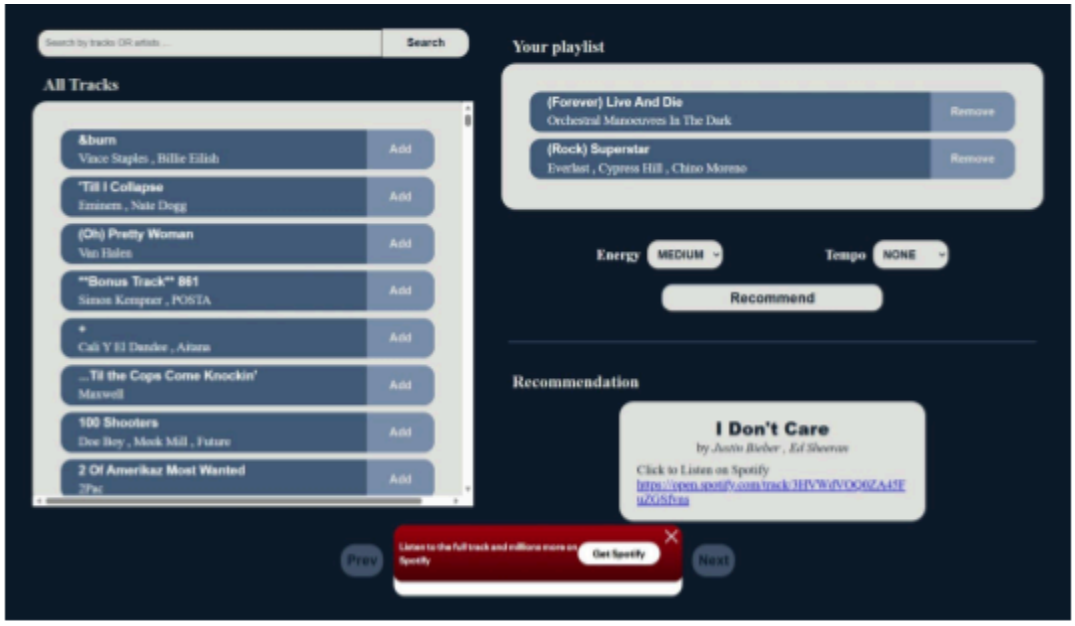
Following the second evaluation, AJAX was introduced using Javascript fetch() requests to enable asynchronous communication between the frontend and backend. Instead of reloading the entire page, Django View endpoints return JSON responses containing updated HTML fragments which are inserted into the page through DOM manipulation.
This approach allows:

- adding and removing tracks without page refresh,
- updating search results in real time,
- saving user preferences asynchronously,
- generating recommendations while maintaining playback continuity.

CSRF tokens are passed through request headers to ensure secure POST requests when updating user preferences.



Frontend Interface of the system (for new users)



Frontend Interface of the system (Large screen)

Frontend Interface of the system (Small screen)

Deployment

The system was deployed on PythonAnywhere. The website can be visited at https://eimyat.pythonanywhere.com/

Evaluation (2119 /2500 words)

Unit Testing

Unit testing was conducted throughout development to ensure the correctness and reliability of individual system components. Tests were implemented for recommender logic, serializers, API endpoints and frontend functions, using model factories and dummy data where appropriate.

Tests for the recommender logic mainly focused on validating the input and output formats, ensuring that valid inputs produced correct outputs, and that invalid inputs raised appropriate errors. Some tests focused on the correctness of mathematical formulas such as Euclidean distance and Cosine similarity, and mathematical operations such as vector multiplications, sorting and getting maximum values. Moreover, the tests also included checking the correctness of database operations such as selecting, excluding tracks and finding artist IDs from track IDs input, with invalid or valid input data. All four different models were also tested to confirm that the correct inputs provide the expected output without duplicate or input tracks in the top-k recommendations.

Serializers test verified that valid data were accepted while invalid inputs were correctly rejected according to defined validation rules

API endpoint testing ensured successful responses for valid requests with or without user preferences, while invalid requests returned appropriate error responses.

Frontend testing focused on checking that the correct url requests with correct data were successful, the frontend functions were working correctly, the correct page was rendered, and the session variables were stored, updated, and deleted as expected.

Overall, the unit tests confirmed that individual functions performed as intended, providing confidence in the correctness of the developed system.

Evaluation on the recommender systems

Objective Evaluation Processes

The recommender models were evaluated in a Jupyter Notebook using the playlists prepared in the Data Preparation section.

For each playlist, tracks were split into 80% input tracks and 20% continuation tracks, with the 20% portion treated as ground truth. The split was performed using Scikit-Learn's train_test_split with random_state of 42 to produce the same split data across evaluation. Each model was queried using the 80% input tracks of each playlist, with the number of recommendations (top-k : 3) passed via the API URL. The returned track IDs were compared with the ground truth to calculate Precision@k, Recall@k and NDCG@k.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall@k = \frac{TP}{TP + FN}$$

$$NDCG@k = \frac{DCG}{IDCG}$$

Precision@k was calculated as the number of relevant tracks in the recommendation list divided by the number of recommendations (k).

```python
# find TP for Precision and Recall and calculate their scores
#  recommended (a list of track ids recommended by the model)
#  expected (a list of track ids from remaining 20% of a playlist)
#  k (the number of recommendation generated by the model)
#  numerator (the relevancy)
#  the denominator (TP + FP <or> TP + FN)
def find_score (recommended, expected, k, numerator, denominator):
    # for every track_id in the recommended list, if that id also exists in expected, the id is relevant id.
    for i in range(min(k, len(recommended))):
        if recommended[i] in expected:
            numerator += 1       # + 1 relevancy

    r = numerator/denominator       # formula
    return r

# PRECISION
#  recommended (a list of track ids recommended by the model)
#  expected (a list of track ids from remaining 20% of a playlist)
#  k (the number of recommendation generated by the model)
def precision_at_k(recommended, expected, k):
    # Precision@k = TP/(TP + FP) <How many of the recommendations are relevant>
    # TP + FP = k
    denominator = min(k, len(recommended))      # handling with min just in case. (length of recommended is surely 10)
    numerator = 0

    return find_score(recommended, expected, k, numerator, denominator)
```

Recall@k was calculated as the number of relevant tracks in the recommendation list divided by the total number of ground truth tracks.

```python
# RECALL
#  recommended (a list of track ids recommended by the model)
#  expected (a list of track ids from remaining 20% of a playlist)
#  k (the number of recommendation generated by the model)
def recall_at_k (recommended, expected, k):
    # recall@k = TP/(TP + FN) <How many of the relevant tracks are predicted>
    # TP + FN = Number of tracks in the expected list
    denominator = len(expected)
    numerator = 0

    if denominator == 0:        #edge case
        return 0.0

    return find_score(recommended, expected, k, numerator, denominator)
```

NCDG@k was calculated using the ratio of DCG@k to IDCG@k. DCG@k was computed by summing 1/ log2(i+1) for each relevant track at rank i, rewarding relevant tracks appearing earlier in the ranked list. IDCG@k was calculated by adding 1/log2(i+1) for maximum possible time, representing the maximum possible DCG value with all relevant tracks being in the ideal rank.

```python
def dcg_at_k (recommended, expected, k):
    dcg = 0.0
    for i in range(min(k, len(recommended))):
        if recommended[i] in expected:      # relevant condition
            dcg += 1 /math.log2(i + 1 + 1)      # adding (index starts from 0, so use (i+1))
            # if irrelevant, the numerator is 0, so, no need to add to dcg.
    return dcg      # final dcg value after all loop

# IDCG@k
#  recommended (a list of track ids recommended by the model)
#  expected (a list of track ids from remaining 20% of a playlist)
#  k (the number of recommendation generated by the model)
def idcg_at_k (expected, k):
    idcg = 0.0
    # if k is 10 and expected (20% playlist) is smaller than that, the best result should be hitting 3 relevant tracks, not 10. so, min between them is used.
    max_hit = min(len(expected), k)
    for i in range(max_hit):      # this is best case, so all are set as relevant,
        idcg += 1 /math.log2(i + 1 + 1)     #so add every time (index starts from 0, so use (i+1))
    return idcg     # final idcg value after all loop

# NDCG@K
# dcg (DCG@K)
# idcg (IDCG@K)
def ndcg_at_k(recommeded, expected, k):
    dcg = dcg_at_k(recommeded, expected, k)
    idcg = idcg_at_k(expected, k)

    if idcg == 0.0:
        return 0.0
    return dcg/idcg    # final NDCG@K
```

Scores were computed for each playlist and averaged across all playlists This process was repeated for all four models, providing a quantitative comparison of their performance.

Subjective Evaluation Processes

Two rounds of user surveys were conducted during iterative development. Surveys collected feedback on recommendation quality, preference effectiveness, and frontend usability, along with suggestions for improvements. This helped improve the system while ensuring alignment with user needs.

Evaluation for first development

Objective Evaluation results

The Euclidean model performed slightly better than the Cosine model, suggesting that Euclidean distance was more suitable for this project. Consequently, the first user survey was conducted using the Euclidean model.

	Model	Precision_First	Recall_First	NDCG_First
0	Euclidean	0.065359	0.041690	0.063424
1	Cosine	0.052288	0.034967	0.054223

Comparison of Euclidean and Cosine model after the first development

Unexpectedly, the random-by-artist model outperformed both the Euclidean and Cosine models. Upon reviewing the evaluation playlists, it was found that some playlists contained multiple tracks by the same artist, reflecting the nature of real-world playlists where users often listen to several songs from the same artist. This result indicates that ‘artist’ is an important feature in track recommendation.

To confirm that the system was functioning correctly, a pure random model was developed and evaluated. Its performance was significantly worse than both the Euclidean and Cosine models, confirming that the recommender system was working effectively with similarity metric, and that the strong performance of the Random-by-artist model was due to dataset bias towards artist similarity.

	Model	Precision	Recall	NDCG
0	Euclidean	0.065359	0.041690	0.063424
1	Cosine	0.052288	0.034967	0.054223
2	Random by Artist	0.398693	0.214683	0.421953
3	Random	0.000000	0.000000	0.000000

Performance comparison of all four models during the first evaluation

Subjective Evaluation results

User feedback from the first survey indicated that the recommender system and preference settings were not performing effectively. Ratings varied widely among participants. The main highlights of this survey was the bad performance of the preference settings, as the system initially applied weightings to the average vector rather than filtering tracks based on user preferences. The Euclidean model recommender itself was reasonably effective despite its poor performance in objective evaluation. Users also noted issues with the music player placement in the frontend, which needed scrolling for small screens.

Please describe your experience with the recommendation quality. In what ways did the recommended song feel similar or different compared to your selected tracks?

4 responses

It feels extremely in tune with my song selection

Yes. Similar rhythm song with the one I choose from the Music Track

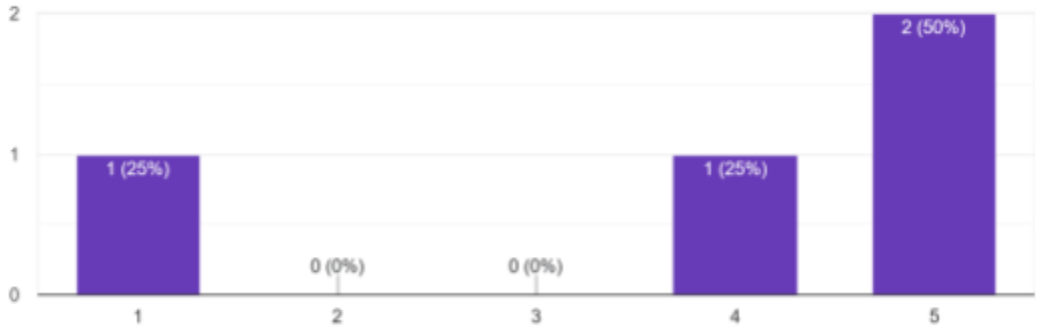
I understand that recommendations can be very subjective. But for Michael Jackson Bad AND Roxette "The Look" I have received something very dancy like "No style" (with selected "low" energy, but to my perspective the energy is very high for the recommended track"). "In real life" with high energy was closer to the selected tracks. But it was recommended no matter did I chose "low" or "medium" tempo which was confusing.

In some songs , when I change the tempo and energy level I still received the same recommendation.

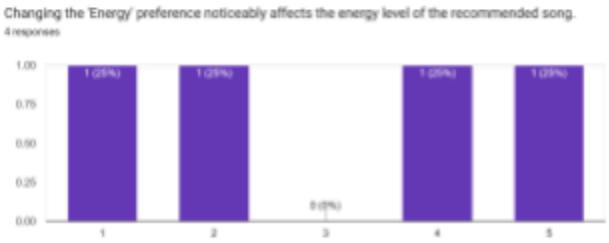
User feedback on recommender model, highlighting the problems with preference settings
(First evaluation survey)

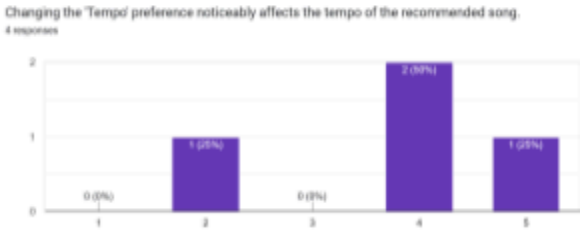
The recommended song is similar in style to my selected tracks.

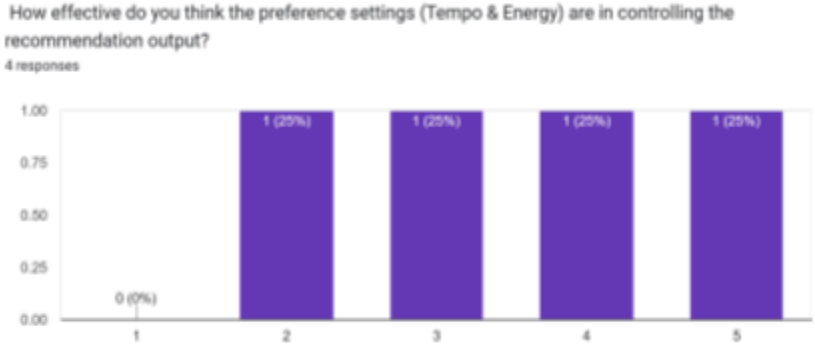
4 responses



Different User feedback on recommender model quality
(First evaluation survey)







User feedback on bad performance of preference settings (First evaluation survey)

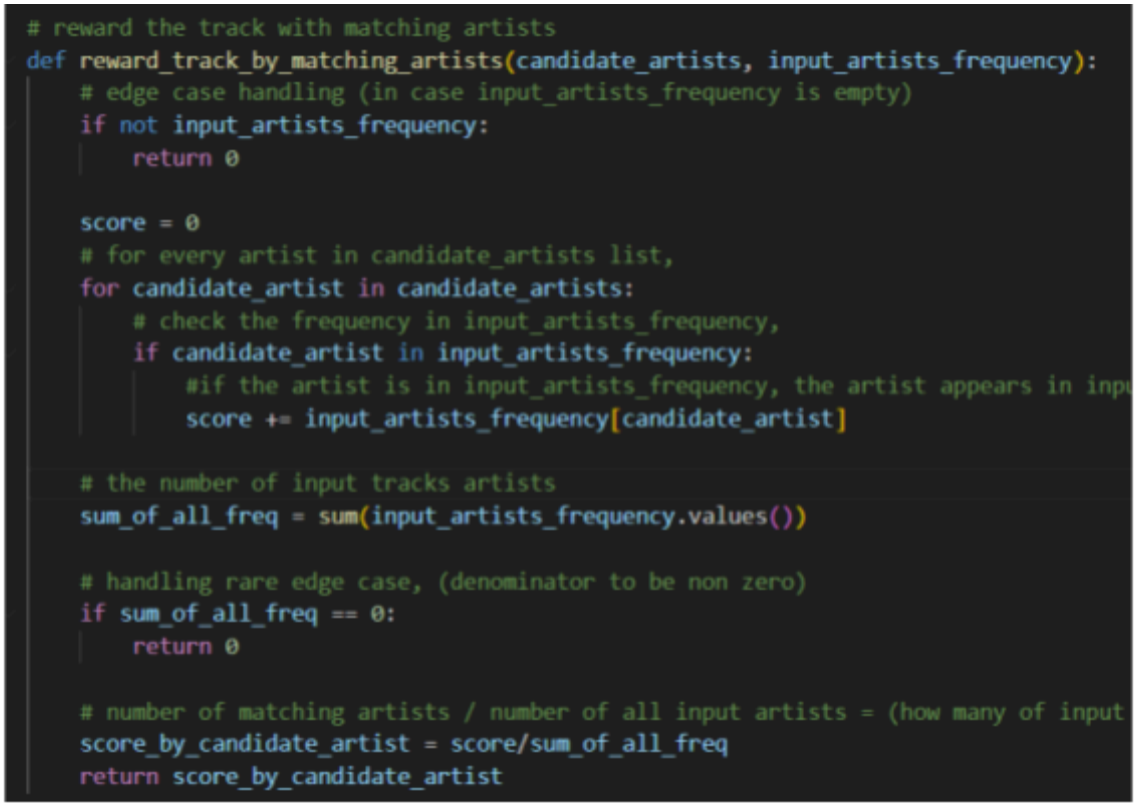
Improvements after the first evaluation

Improvement 1

The objective evaluation indicated that artist is an important feature, Three approaches were considered for incorporating artist similarity:

- Filtering tracks by artist
    - It was rejected because it would ignore tracks with different artists, limiting artist diversity.
- One hot encoding vector for the artist category
    - It is common in machine learning, but it was not used due to time limitation as it would require reprocessing the dataset from the scratch
- Rewarding tracks with matching artists
    - It was selected as the most practical and effective solution

Artist similarity was implemented by calculating how many artists of the specific track are included in the input artists.



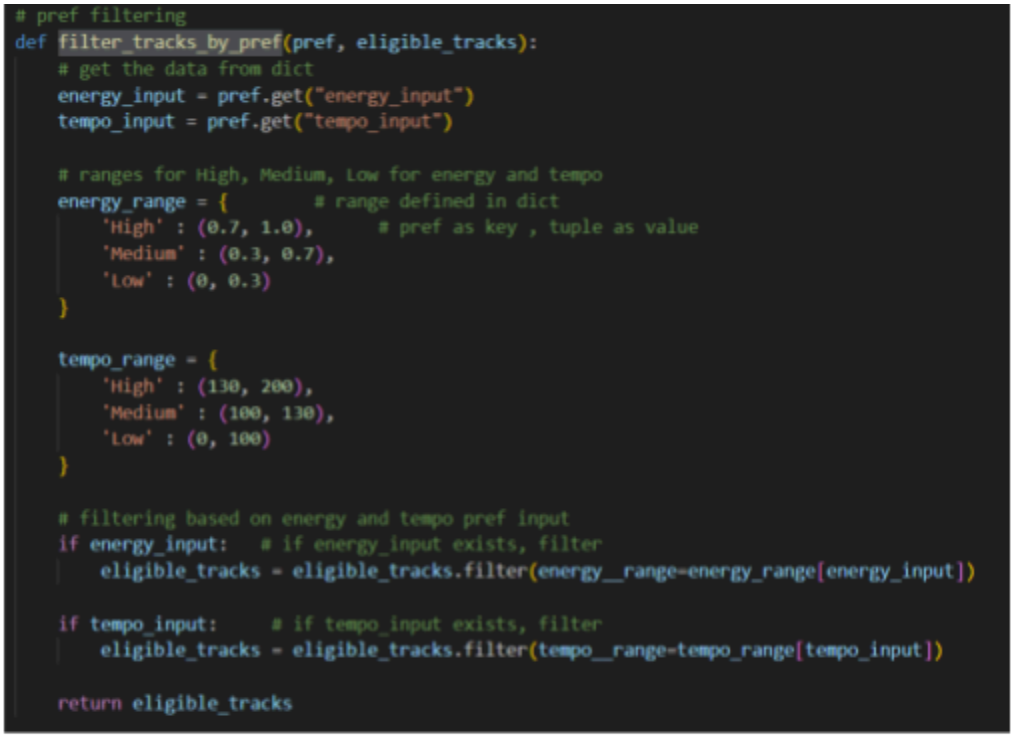
```python
# reward the track with matching artists
def reward_track_by_matching_artists(candidate_artists, input_artists_frequency):
    # edge case handling (in case input_artists_frequency is empty)
    if not input_artists_frequency:
        return 0

    score = 0
    # for every artist in candidate_artists list,
    for candidate_artist in candidate_artists:
        # check the frequency in input_artists_frequency,
        if candidate_artist in input_artists_frequency:
            #if the artist is in input_artists_frequency, the artist appears in inpu
            score += input_artists_frequency[candidate_artist]

    # the number of input tracks artists
    sum_of_all_freq = sum(input_artists_frequency.values())

    # handling rare edge case, (denominator to be non zero)
    if sum_of_all_freq == 0:
        return 0

    # number of matching artists / number of all input artists = (how many of input
    score_by_candidate_artist = score/sum_of_all_freq
    return score_by_candidate_artist
```

The implementation of Artist Similarity

This score was combined with the original similarity score to produce an overall similarity score for recommendations.
Determining the weighting for each similarity score was challenging because low artist similarity weighting only produced minimal improvements. Although the model with an 80% artist similarity weighting performed better than the Random-by-artist model, it was not reasonable to assign such a big weighting arbitrarily. Therefore, even though the results of the Cosine and Euclidean model did not surpass the Random-by-artist model, 25% weighting for artist similarity and 75% for feature-based similarity was chosen to balance effectiveness and interpretability of the model.

Improvement 2

Regarding the bad performance of preference settings, the first user survey highlighted a difference between developer assumptions and user expectations. Initially, I applied respective weightings of preferences to the average vector. However, users expected that selecting “Low Energy” would result in low-energy tracks, rather than a subtle numerical weighting difference. Based on this feedback, the system was updated to filter tracks according to the user input preferences, ensuring the recommendations better matched user intent.



```python
# pref filtering
def filter_tracks_by_pref(pref, eligible_tracks):
    # get the data from dict
    energy_input = pref.get("energy_input")
    tempo_input = pref.get("tempo_input")

    # ranges for High, Medium, Low for energy and tempo
    energy_range = {        # range defined in dict
        'High' : (0.7, 1.0),      # pref as key , tuple as value
        'Medium' : (0.3, 0.7),
        'Low' : (0, 0.3)
    }

    tempo_range = {
        'High' : (130, 200),
        'Medium' : (100, 130),
        'Low' : (0, 100)
    }

    # filtering based on energy and tempo pref input
    if energy_input:    # if energy_input exists, filter
        eligible_tracks = eligible_tracks.filter(energy__range=energy_range[energy_input])

    if tempo_input:     # if tempo_input exists, filter
        eligible_tracks = eligible_tracks.filter(tempo__range=tempo_range[tempo_input])

    return eligible_tracks
```

Filtering the tracks based on preferences

Improvement 3

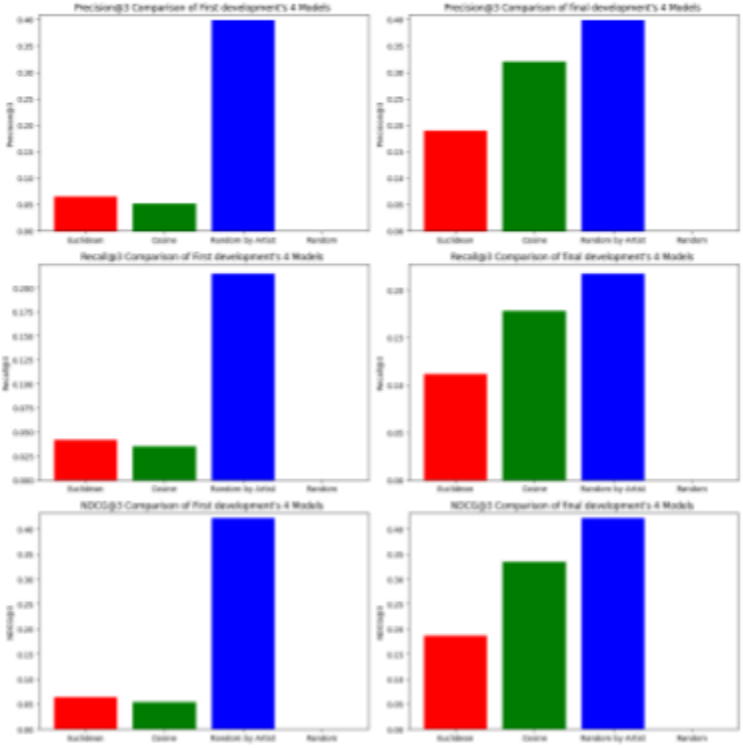
The frontend usability was improved by making the music player float, instead of displaying normally between the sections, to ensure that it remained visible without requiring users to scroll.

Evaluation for Second development

Objective Evaluation Results

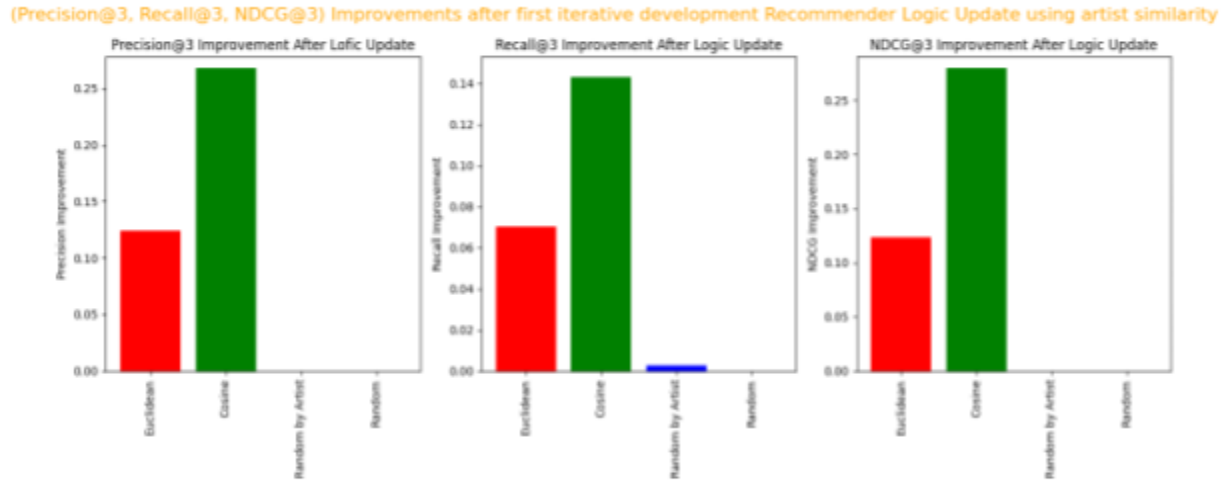
The Precision, Recall and NCDG scores of the Euclidean and Cosine models showed significant improvements over the first development. While the results did not surpass the performance of the Random-by-artists model, they were very close to each other, indicating that

the updated logic provided more meaningful recommendations.



Comparison of different recommender models before and after the logic improvements

After the improvements of recommender logic with artist similarity, the Cosine model's performance significantly improved, further surpassing the 'Euclidean' model and nearly reaching the 'Random-by-artist' model performance. This indicated that the 'Cosine' model is a better model than the 'Euclidean' model. Therefore, the 'Cosine' model was used as the main model in the second user survey.

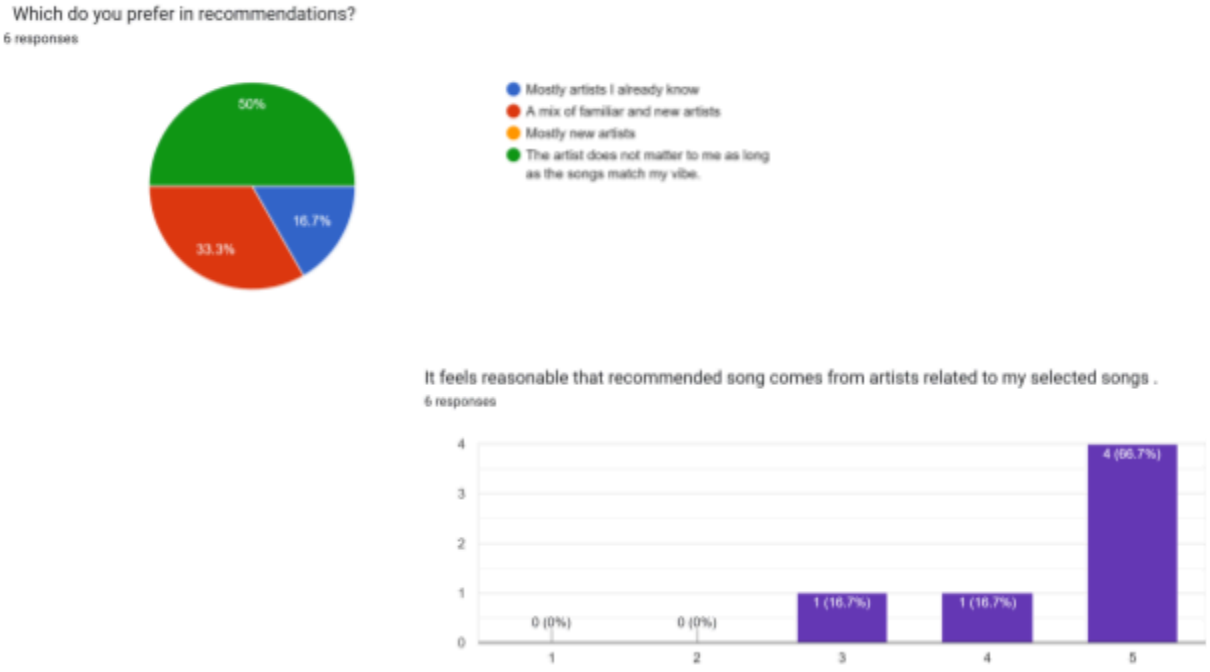


Comparison of four models' improvements after the recommender logic update

Subjective Evaluation Results

Although incorporating artist similarity improved objective evaluation, it introduced a potential risk of reduced diversity, as tracks from frequently occurring artists could dominate recommendations. To investigate this, the second user survey included questions about user preferences regarding artist influence

Survey results showed that 50% of users stated that the artist does not matter as long as the songs match their preferred vibe, while 33.3% preferred a mix of familiar and new artists, and only 16.7% preferred mostly familiar artists. Moreover, two-thirds of the users felt it was reasonable for recommended tracks to come from artists related to their selected songs.



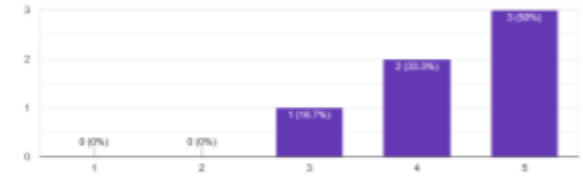
User opinion on artist similarity

These findings indicate that users primarily value musical similarity rather than artist familiarity, although recommendations from related artists are still considered acceptable. Consequently, artist similarity was treated as a supporting factor rather than a dominant feature in the recommendation process. The selected weighting of 25% artist similarity and 75% feature similarity appropriately balanced familiarity with diversity while maintaining focus on track characteristics.

User feedback also indicated improvements in recommender quality and preference settings, with some users noting that the system's recommendations had noticeably improved.

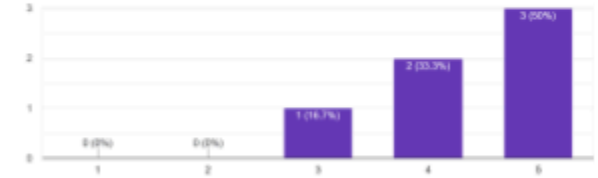
The recommended song is similar in style to my selected tracks, especially with preference settings of "None".

6 responses



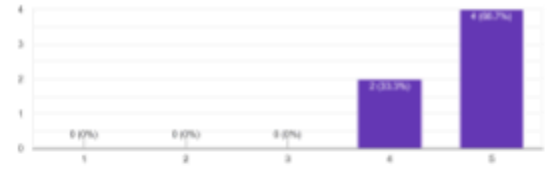
The recommended song is similar in style to my selected tracks, especially with preference settings of "None".

6 responses



How effective do you think the preference settings (Tempo & Energy) are in controlling the recommendation output?

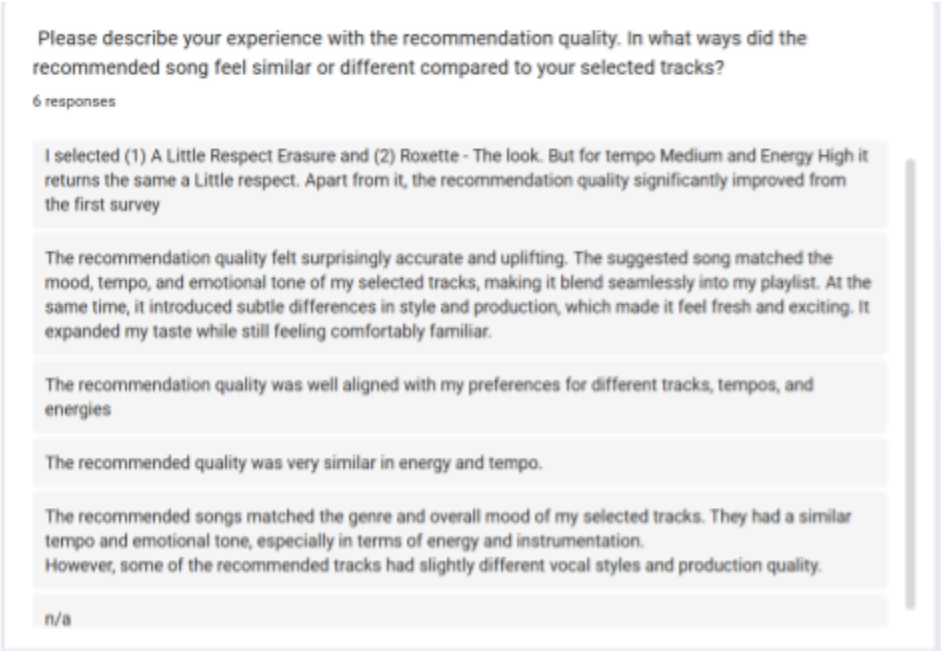
6 responses



Overall, I am satisfied with recommender quality.

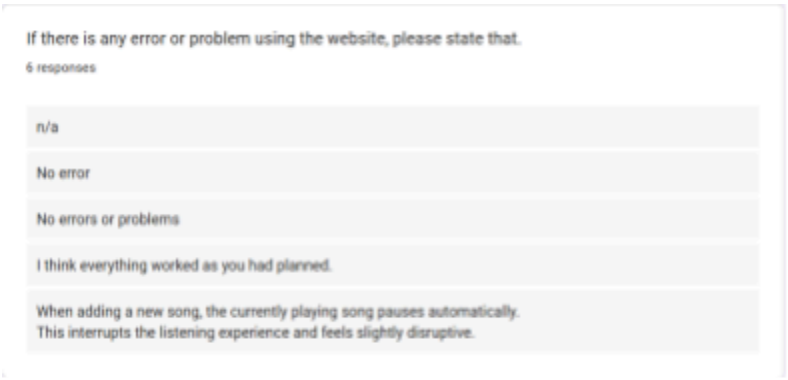
6 responses

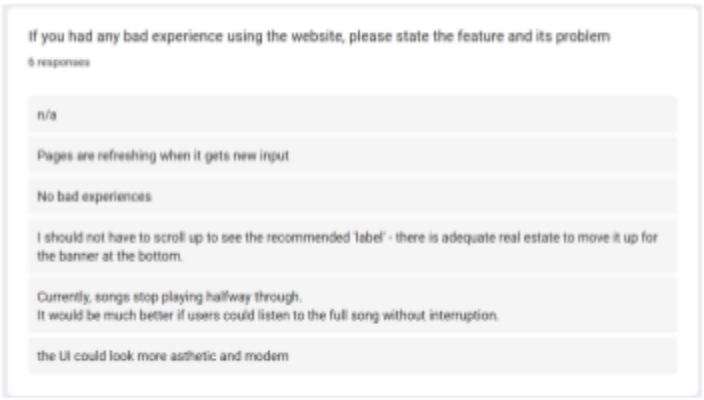
User rating on the recommendation quality and preference settings (Second evaluation survey)



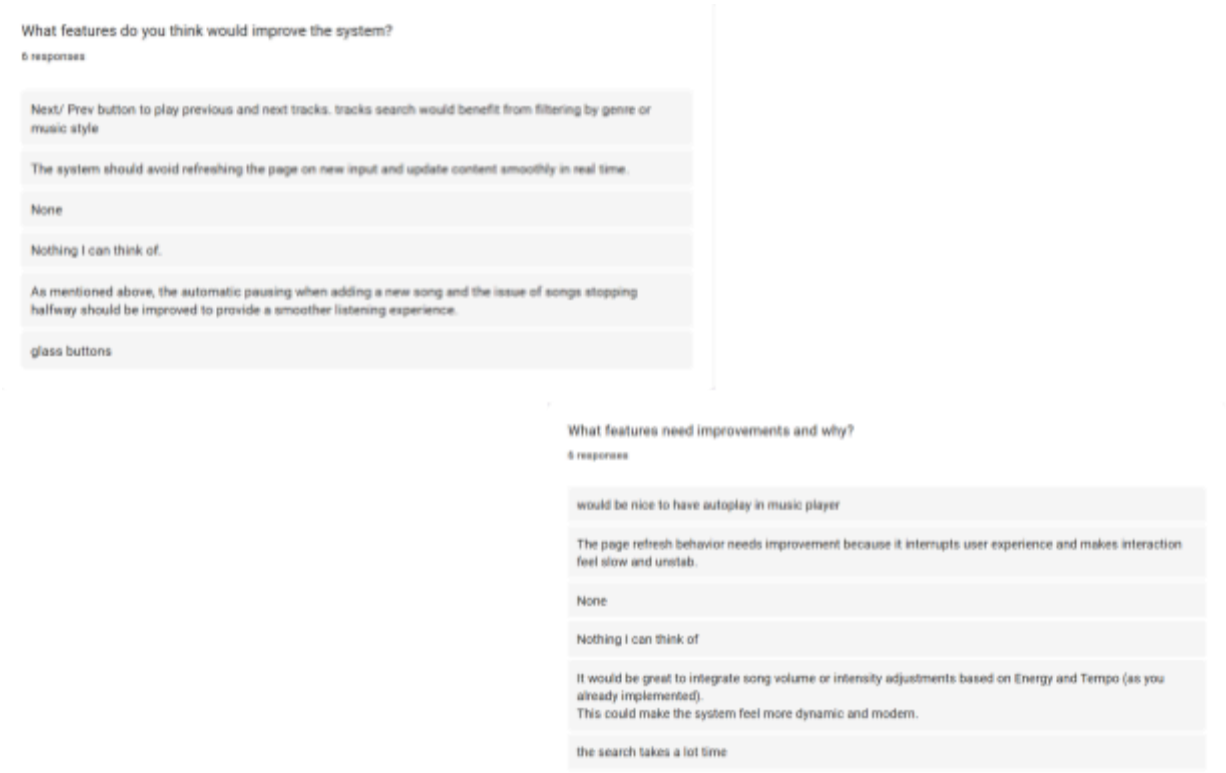
User feedback on the recommendation quality and preference settings (Second evaluation survey)

However, many responses suggested enhancements to the UI, including adding Next and Prev buttons for the music player, improving search function with filtering options, and ensuring the recommendation section is visible without excessive scrolling. Some users also reported frustration with the music stopping or pausing unexpectedly when interacting with the page, highlighting the need for smoother, uninterrupted webpage.





User feedback based on their experience using the system and suggestions for the system
(Second evaluation survey)



User feedback and suggestions for system improvements (Second evaluation survey)

Improvements after second evaluation

Improvement 1

With recommendation quality and preference settings improved during the second evaluation, the focus shifted to enhancing the frontend based on user feedback. Due to time constraints, priority was given to addressing the most essential features that would improve user experience. Many users reported that the page refresh behaviour and the music player stopping during interactions were disruptive. To address this, AJAX was implemented to allow real-time HTML content updates, preventing full page refreshes and minimizing interruptions to playback.

Improvement 2

Next and Prev buttons were implemented to allow users to navigate the playing history during the session. Additionally, minor layout adjustments were also made so that the recommendation section is at least partially visible by default, improving the accessibility and reducing the need for scrolling.

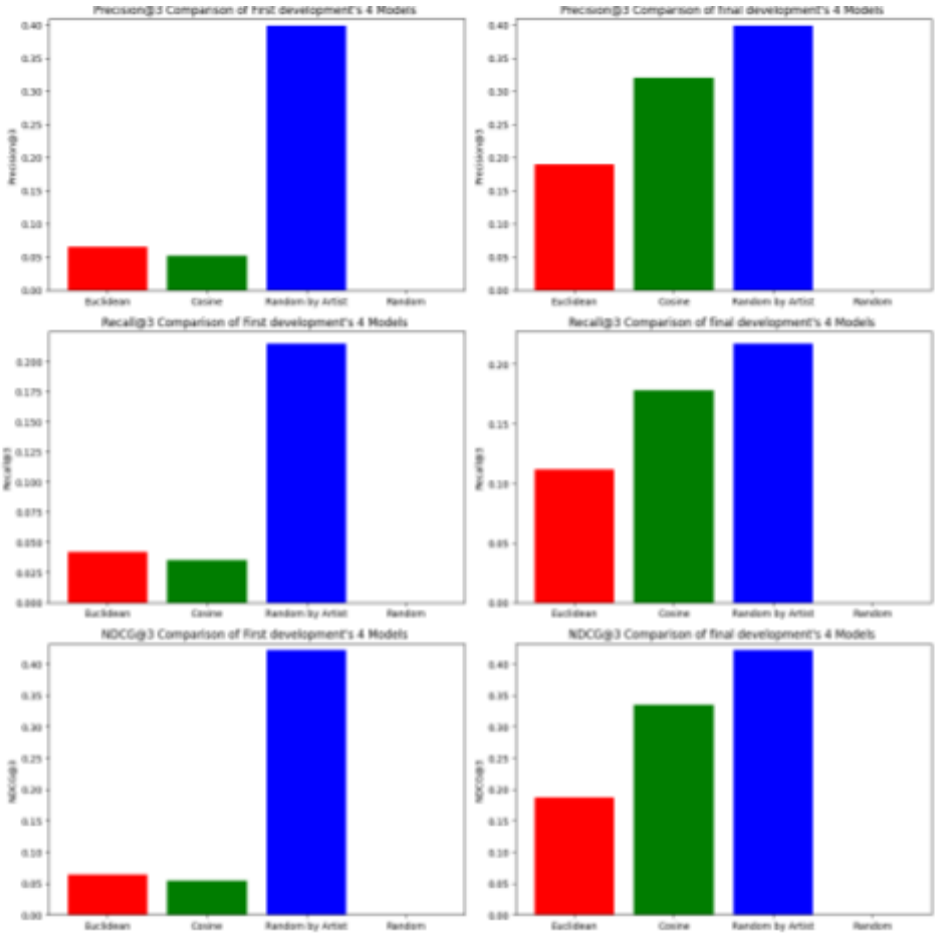
These improvements directly addressed user concerns and enhanced the overall responsiveness and usability of the music recommender system.

Future improvements

As time limits, the other suggestions such as implementing a more modern UI and adding filtering options for search, were acknowledged and will be considered in future iterative development cycles.

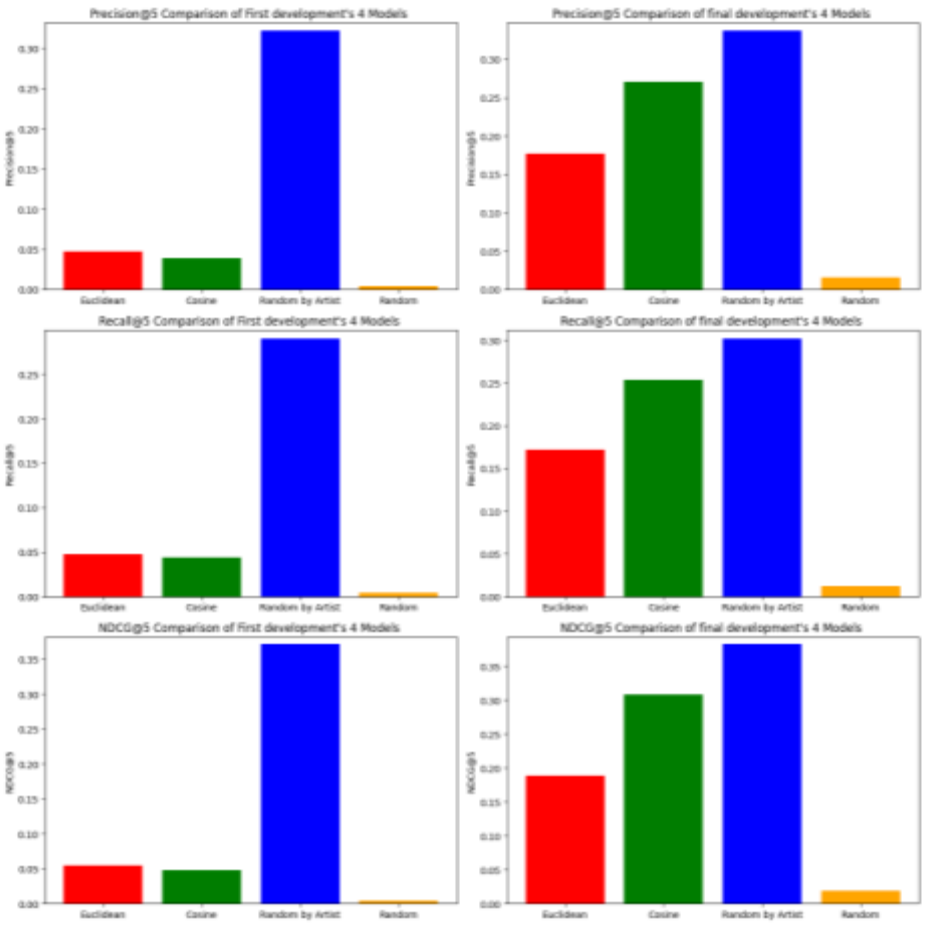
Model Comparisons & Additional Objective Evaluation

During the initial development, evaluation was conducted using the top 5, 10, and 15 recommended tracks. However, in the second iteration, it became clear that selecting an appropriate top-k value is important. The evaluation playlists contained only 15 to 40 tracks, with only 3-8 tracks (20%) considered as ground truth. Therefore, using k = 10 or 15 produced unreliable results. To ensure more meaningful evaluation, only the top 3 and top 5 recommended tracks were used in the second iteration.
The full evaluation notebook is available on GitHub: Model Evaluation & Comparisons of model performances
The figure below shows the Precision@3, Recall@3, and NDCG@3 values of the previous model (developed before the first evaluation) and the current model (developed after the logic improvements). In the first version, the main models (Euclidean and Cosine similarity) underperformed compared to the Random-by-artist baseline. After incorporating artist similarity into the recommender logic, both Euclidean and Cosine models showed substantial improvement, with the Cosine model's results nearly matching the Random-by-artist model.



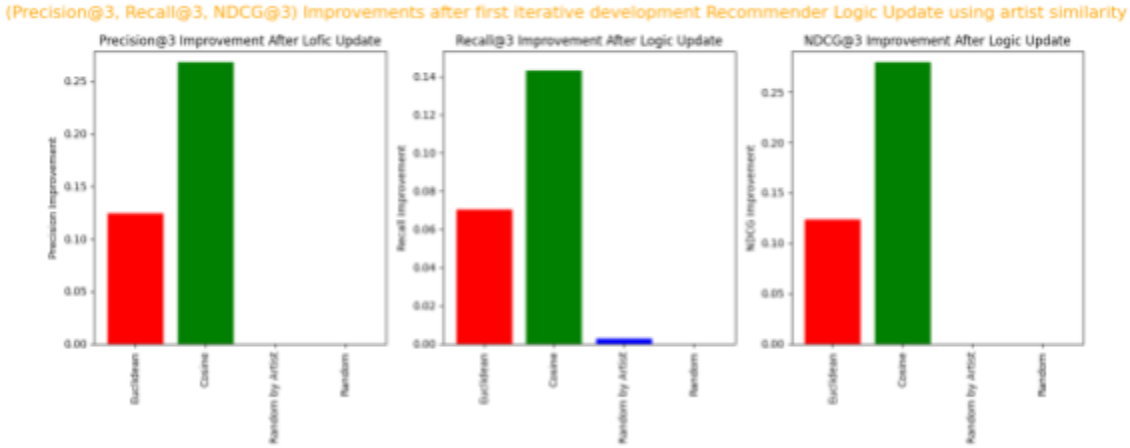
(Top 3) Performance comparison of four different models before and after the logic improvements

The Precision@5, Recall@5 and NDCG@5 values followed a similar trend as the top-3 results but were slightly lower. This is likely due to the k value sometimes being larger than the number of available ground truth tracks in the evaluation playlists.



(Top 5) Performance comparison of four different models before and after the logic improvements

The score differences of Euclidean and Cosine models before and after the logic improvements highlight the significant enhancement in both models. Notably, the Cosine model improved roughly twice as much as the Euclidean model, likely because Cosine similarity benefits more from the artist similarity addition. This may be due to the difference between the nature of the two formulas as Euclidean distance emphasizes feature magnitudes while Cosine similarity emphasizes the angle between feature vectors.



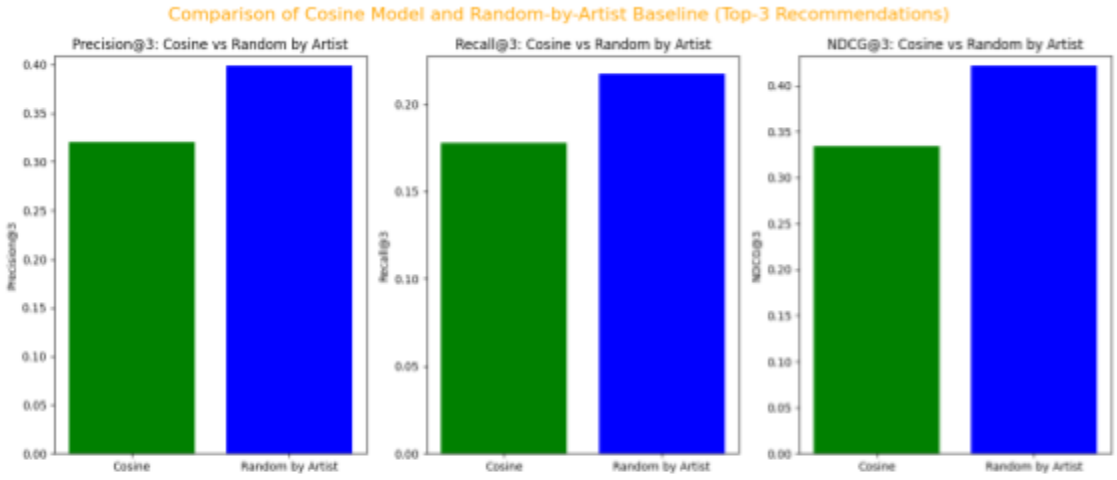
The improvements of Euclidean, Cosine and other models after the logic improvement

Interestingly, before the improvements, the Euclidean model slightly outperformed the Cosine model, although both performed poorly overall. After the improvements, the Cosine model demonstrated clear advantage, significantly surpassing the Euclidean model. Consequently, despite initially planning to use Euclidean as the main model, the Cosine model was selected as the main recommender after the second evaluation.

Performance comparison of Euclidean and Cosine models before and after the improvements

The comparison between the Cosine model and the Random-by-artist model is shown below. The Random-by-artist baseline performed the best among all models, both before and after the logic updates. However, it was not chosen as the main model because it was intended as a baseline and relies solely on the input tracks' artists. In contrast, the Cosine model balances both feature similarity and artist similarity, making it a more reliable personalized recommender. Moreover, the difference between the Cosine and the Random-by-artist model was not too big

after the logic updates. Therefore, I did not want to use the Random-by-artist model as it is random and artist-based, and only used the Cosine model as my main model. Additionally, the second subjective evaluation results using the Cosine model were really positive, and therefore, supported the effectiveness of the Cosine model as the main recommender model.



Comparison of (current) Cosine and Random-by-artist models

Conclusion (837/1000 words)

The project showcases the implementation of a personalized music recommender system and its API endpoint, based on audio features' similarity using the content-based filtering approach. The aim was to provide meaningful recommendations without relying on user tracking.

Four different recommender models (Cosine, Euclidean, Random-by-artist, and Random) were developed with unit tests, and evaluated using playlist-based evaluation with Precision@k, Recall@k, and NDCG@k metrics to determine the most effective model. The recommendation quality, preference settings and user interface were also evaluated through the user feedback. Using both subjective and objective evaluation results and further analysis, the system was iteratively developed and improved to get the current recommender model with positive user feedback and good precision, recall and NDCG scores. During the iterative developments, the system was improved with better recommendation logic using both feature-based similarity and artist similarity. Moreover, the preference setting logic was changed from applying weightings to filtering tracks based on user preferences. Many improvements to the frontend interface such as adjusting layouts, changing the music player location, making the website asynchronous to avoid disruptive page reloads, and adding the next and previous buttons for music player, were also made to develop a better system.

Problems faced throughout the project

Throughout the project, I faced a lot of difficulties. Firstly, it was difficult to find the right playlist dataset with the appropriate audio features and track data, but fortunately, I managed to find the appropriate dataset. The project was initially planned to use Spotify API for fetching audio features, but during the prototype development, I realized that the API was deprecated and couldn't be used, therefore, the plan had to be revised, changed and new functionalities such as the search function were added.
Moreover, during the first evaluation, both the objective evaluation results and the survey feedback were unexpectedly negative, which caused me considerable concern about the system's effectiveness. However, I managed to find the logic concerns and improve the recommender system to get better subjective and objective evaluation results.
Additionally, there were also times when I did not fully understand the evaluation metrics. I only realized that I should use an appropriate top-k value based on the evaluation dataset size during the second evaluation, and therefore, managed to change the top-k value later and do the appropriate model comparisons and analysis.
Overall, although the project development encountered multiple problems, appropriate solutions were implemented to overcome them.

Reflection & Lesson learned

During the developments, I have known important software development practices. Unit tests are very useful in checking the code correctness and reliability. This was more obvious in this project because the system mainly focuses on the correctness of the recommender logic which includes mathematical operations. Therefore, the recommendation correctness and code reliability was checked using the unit tests.
I have also learned that the research phase is important to gain domain knowledge to plan the system design. Therefore, it is important to research carefully to effectively design the project so that the project can be built as planned. During the project, even though I had researched that the Spotify API can fetch the audio features, I had not known that it was deprecated until I built the prototype. Moreover, I learnt that utilizing an appropriate dataset is very important for machine learning related projects which need objective evaluation. It is also important to fully understand the evaluation methods and metrics, before performing evaluation, to get reliable evaluation results.
Last but not least, I have realized the importance of user feedback and user centered designs in this project as I noticed that the developer's perspective and user's perspective can be different during the first survey.

In summary, if I were to start the project again, I would do more research on one-hot encoding techniques, evaluation methods and metrics at an earlier stage. I would also put more effort into finding the appropriate dataset before implementation to reduce evaluation challenges.

Future Improvements

In the future, I will conduct further iterative development using additional user surveys, and objective evaluation to continuously improve the system. As the survey suggested, the frontend interface can be enhanced with a more modern design, and search filtering functions can be added to improve usability. Moreover, I want to study one-hot encoding techniques to investigate whether representing artist information and genre as categorical vectors can improve recommendation performance compared to the current artist similarity approach. I will also populate the dataset with more tracks to increase the track pool.

In conclusion, this project demonstrates the successful design and iterative development of a personalized music recommender system that achieved the project aim of providing meaningful recommendations without relying on user tracking. Through continuous evaluation, user feedback, and improvements, the system became an effective and user-centered application. The project highlights the importance of unit tests, evaluations, and iterative developments to build a software system. Overall, the development was a valuable real-world learning experience for me, as I was involved in every stage of development, including background research, planning, prototype development, iterative development, evaluation, and deployment.

References

[1] Niyazov, A., Mikhailova, E., and Egorova, O. 2021. Content-Based Music Recommendation System. *In Proceedings of the 29th Conference of Open Innovations Association (FRUCT)*. FRUCT, 2021, IEEE, 274–279. https://doi.org/10.23919/FRUCT52173.2021.9435533

[2] Eliyas, S., and Ranjana, P. 2022. Recommendation Systems: Content-Based Filtering vs Collaborative Filtering. *In Proceedings of the 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), 2022*, IEEE, 1360-1365. https://ieeexplore.ieee.org/document/9823730/

[3] Bogdanov, D., et al. 2009. From Low-Level to High-Level: Comparative Study of Music Similarity Measures. *In Proceedings of the IEEE International Symposium on Multimedia*, *2009,* IEEE, 453–458. https://ieeexplore.ieee.org/document/5366050/

[4] Zeng, T., and Umrawal, A. K. 2025. Content filtering methods for music recommendation: A review. arXiv:2507.02282. Retrieved from https://arxiv.org/abs/2507.02282.

[5] Panda, R., Malheiro, R., and Paiva, R. P. 2023. Audio Features for Music Emotion Recognition: A Survey. *IEEE transactions on affective computing* 14,1 (2023), 68–88. https://ieeexplore.ieee.org/document/9229494/

[6] Spotify Developers. 2025. Get Audio Features for a Track. Spotify Web API. Retrieved Nov 2025 from https://developer.spotify.com/documentation/web-api/reference/get-audio-features

[7] Vall, A., et al. 2019. Feature-Combination Hybrid Recommender Systems for Automated Music Playlist Continuation. *User modeling and user-adapted interaction* 29, 2 (2019), 527–572. https://doi.org/10.1007/s11257-018-9215-8